CacheForge (FunSearch) Framework Project Report

Varad Patwardhan, David Mond, Obblivignes KanchanadeviVenkataraman,
Jaxon Godbey, Ravi Pavuluri, and Inyene Etuk
North Carolina State University
Departments of CSC, CPE, and ECE

***Abstract*—Developing efficient cache replacement policies are paramount towards building faster computer systems. However, building such a policy that effectively beats existing SOTA policies against a variety of different workloads can prove extremely difficult. In recent days, using Large Language Models (LLMs) in an evolutionary workflow have been successfully implemented to develop new pharmaceuticals, create efficient solutions for mathematical problems, and discover unknown scientific phenomena. In this final project, we will describe our approach to create an LLM-generated evolutionary workflow that can successfully generate consistently better policies with three important features: (1) graph-of-thought to save prior iterations and guide decision making, (2) surrogate model to accurately determine the best policy from n-generated candidate policies, and (3) an intelligent search strategy that balances lineage diversity, performance-guided refinement, and novelty injection. Applying these changes has been successfully shown to improve overall instructions per cycle (IPC) from current baselines with limited area required.**

***Index Terms*—LLM-in-the-loop framework, cache replacement policy, graph-of-thought, surrogate model, IPC**

I. Introduction

Modern computer systems are fundamentally designed to tackle three major tasks: data storage main operations: data storage/retrieval and data manipulation. To ensure efficient interaction between these operations, a hierarchy of memory abstractions has been developed, including persistent storage, main memory (RAM), and the central processing unit (CPU).

Cache memory, positioned between the CPU and main memory, plays a critical role in bridging the speed gap by storing and evicting the best blocks. Organized as a two-dimensional grid of sets and ways, each cache block holds a contiguous segment of data from RAM. Memory addresses are decomposed into offset, index, and tag bits to facilitate mapping into cache sets. Because cache capacity is limited, replacement policies are required to determine which blocks to evict when new data must be loaded.

Designing high-performance cache replacement policies has historically required deep domain expertise and extensive manual tuning. Traditional policies such as LRU, re-reference interval prediction (RRIP) [1], and signature-based hit predictor (SHiP) [2] encode fixed heuristics that capture broad patterns but struggle to generalize across diverse memory access behaviors found in large benchmark suites like SPEC CPU 2006. To address this gap, we decided to utilize the recent technology called large language models, which are extremely large machine learning models trained on vast datasets of human natural language to answer text-based questions with an equivalent text-based answer. These techniques have been successfully demonstrated to synthesize novel pharmaceutical discoveries, identify scientific phenomena, and create efficient mathematical solutions. As such, we consider utilizing LLMs to synthesize new and improve upon existing cache replacement policies such that IPC and other metrics are improved upon existing workloads in large benchmarks.

Specifically, our project builds on several existing ideas towards LLM frameworks that we will utilize towards creating effective cache replacement policies with limited resources. First, FunSearch [3] has successfully demonstrated that LLMs could be embedded inside an evolutionary loop (otherwise called as LLM-in-the-loop framework) to iteratively refine algorithmic artifacts while guided by an external evaluator. Instead of directly learning the function, the model learns how to generate better candidates over time. Graph-of-Thought [4] devises an entire graph-like memory architecture designed to help guide an LLM remember previous iterations, and decide whether it is best to (1) improve upon existing methods, (2) generate new candidate policies from existing methods, or (3) aggregate existing solutions to generate a final candidate. Finally, automated prompt engineering (APE) [5] has been successfully shown to help the LLM generate specialized prompts based on prior iterations and search strategies. We will build these ideas off of an existing proprietary cache replacement policy generation framework called CacheForge.

Our objective is to integrate three separate components into this LLM-in-the-loop framework that we will be used to create an experience-driven search system, which are (1) structured Graph-of-Thought (GoT) based memory, (2) automated prompt engineering to create unique specialized prompts for each iteration, and (3) a learned surrogate reward model to efficiently determine best policies to move forward. We believe that this dedicated system will allow itself to learn from its history and uses this accumulated experience to guide future exploration. This approach dramatically reduces repeated mistakes, improves sample efficiency, and enables scalable exploration across the full SPEC CPU 2006 suite. We will discuss the full details regarding the relevant frameworks and strategies in Section II below. Ultimately, our goal is to demonstrate that LLMs, when paired with persistent memory and predictive modeling, can autonomously synthesize policies that outperform classical heuristics while respecting strict metadata constraints. In doing so, we showcase a pathway toward using a complex LLM-driven discovery framework to solve difficult systems design problems that are traditionally solved by manual engineering and simplistic LLM-in-the-loop

frameworks.

II. Background

Before discussing the related papers that we will be considering for our evolutionary search algorithm built from CacheForge, it is best to understand the relevant context for determining a good cache replacement policy, including relevant performance metrics that we will evaluate cache replacement policies by, as well as relevant baseline cache replacement policies that we will be judging LLM-generated policies against.

A. Performance Metrics

To best evaluate the performance of a given cache replacement policy, we will utilize two main metrics, which we will discuss in detail below.

1) Cache Hit Rate: The cache hit rate is defined as the ratio of cache hits (or when data is found in the cache) to the total number of cache accesses.

Ideally, an optimal cache hit rate of 1 would imply that the cache will always have the necessary cache blocks that would need to be accessed at any given time, but this is almost never guaranteed. As such, a higher cache hit rate generally indicates a better performance for the specific cache replacement policy.

2) Instructions Per Cycle: Another common metric used to evaluate the performance of a cache replacement policy is the instructions per cycle or IPC in short. As its name suggests, this measures how many instructions the CPU can process for each cycle (instructions executed per unit of time). Although this factor is heavily determined by multiple factors, including prefetching, branch prediction, pipelining, thermal throttling, and more, improving the cache replacement policy to maximize IPC can dramatically improve the processor's speed and capabilities.

In our evaluation, IPC will serve as a comprehensive measure for policy effectiveness, as a high cache hit rate alone does not guarantee good performance across diverse workloads. However, when comparing overall performance, we cannot simply use the arithmetic mean as it will overweight workloads with higher IPC values. As such, we will use the harmonic mean, which will equally balance contribution from each workload fairly. For reference, both of these equations have been defined below.

$$A = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \text{(Arithmetic Mean)}$$

$$H = \frac{n}{\sum_{i=1}^{n} \frac{1}{x_i}} \qquad \text{(Harmonic Mean)}$$

B. Cache Replacement Policies

There are many existing cache replacement policies that all aim to maximize the performance metrics that we described earlier. Common baseline policies include least-recently-used (LRU) and Random. At the same time, there are several state-of-the-art policies that have been designed to improve upon these baselines, which can be categorized towards optimizing on heuristis, prior knowledge, or machine learning. We will discuss each approach in brief below.

1) Baselines: One of the most widely used cache replacement policies is the *Least Recently Used (LRU)* policy, which assumes that policy assumes that data used more recently is more likely to be used again. When the cache is full, LRU will always remove the least recently used block. Another common baseline policy is *Random*, which, as the name suggests, selects a random cache block to evict, is quite simple, but does not provide any performance guarantees. Finally, the *Reordering* cache replacement policy rearranges cache lines when a line is inserted such that frequently used lines remain present in the cache for longer periods.

2) Heuristics-based policies: Some of the earliest and efficient policies are heuristics-based, meaning they rely on heuristics or approximations of optimal behavior rather than exact solutions. As discussed earlier, LRU is one of the simplest versions of this type of policy. Another approach relies on Belady's optimal cache replacement policy, which predicts the cache line that will be reused farthest in the future. Several modern policies build on this principle, including the *Dynamic Insertion Policy (DIP)* [6] that uses insertion strategies to adapt between different workloads, *Re-reference Interval Prediction (RRIP)* [1] which predicts how soon a cache line will be reused, and *Signature-based Hit Protector (SHiP)* [2], that associates each cache block with instruction signatures based on program counters (PCs) to track reuse history across instructions.

3) Learning-based policies: Learning-based (or predictive) policies build further from heuristic-based policies, using prior knowledge, such as historical access patterns and program behavior, to help guide their decision towards what blocks to evict at any given moment, which enables them to adapt dynamically on decision-making capabilities based on workloads, tasks, or environments. One such policy that falls in this category is *Hawkeye* [7], an adaptation of Belady's optimal algorithm that evicts the block that will be used the farthest in the future. Some other policies include Glider [8], which modifies Hawkeye to optimize for better handling of prefetches through the use of program counter (PC)-based predictors; *Less is More (LiM)* [9], which selects caches only the most promising memory blocks through multiple features like PC, access type, and reuse history through the guidance of Belady's algorithm and a binary classifier; and *PARROT* [8], which further trains on Belady's algorithm to create a predictive policy using time-series cache access history.

4) Machine Learning-based policies: Further developments into learning-based policies have resulted in machine learning (ML)-based policies, which utilize intelligent models to capture cache reuse behavior with minimal overhead. For example, the *Perceptron Branch Indicator* [10] successfully demonstrated that a simple multi-layer perceptron model can be efficiently applied towards computer hardware. *Multiperspective Reuse Prediction* [11] leverages multiple signals, such as recency, program control, and address bits, to determine whether a cache line should be promoted or bypassed from the cache. Other ML-based algorithms have also been used to develop good cache replacement policies, including genetic search algortihms and reinforcement learning techniques, such as *Reinforcement Learned Replacement (RLR)* [12] and *Storm-*

bird [13].

For our evaluations, we will be using six different baseline methods to identify the best baseline amongst given workloads and compare with LLM-generated policies, which are LRU, Less is More, Multiperspective, and Reordering, SHiP, and Hawkeye.

C. Workloads

Although there are many workloads available to test the performance of any given cache replacement policy, we will be testing all cache replacement policies on ten different common workloads that are part of the SPEC 2006 CPU benchmark suite. We have indicated the specific workload descriptions and individual characteristics of each workload in the ordered list below.

1) **403.gcc** [14]: Simulates the compilation of C code into x86 assembly. Characteristics include a large instruction footprint, complex control flow, and irregular data access patterns driven by syntax tree traversals.
2) **429.mcf** [15]: Simulates a vehicle scheduling algorithm using a network simplex solver. Characteristics include heavy pointer chasing, poor spatial locality, and a very high rate of Last-Level Cache (LLC) misses, making it highly sensitive to cache capacity and replacement decisions.
3) **433.milc** [16]: Simulates lattice gauge theory (QCD) for quark and gluon dynamics. Characteristics include regular strided memory access patterns and high demand for memory bandwidth.
4) **434.zeusmp** [17]: Simulates astrophysical phenomena using computational fluid dynamics (CFD). Characteristics include streaming memory access patterns over large arrays, stressing memory bandwidth.
5) **444.namd** [18]: Simulates large biomolecular systems using classical molecular dynamics. Characteristics include being largely compute-bound with relatively stable loop structures and good spatial locality compared to other benchmarks.
6) **454.calculix** [19]: Simulates linear and nonlinear 3D structural mechanics using Finite Element Method (FEM). Characteristics include a mix of indirect memory addressing and dense matrix operations.
7) **456.hmmer** [20]: Simulates search across a gene sequence database using Hidden Markov Models (HMMs). Characteristics include high integer computation intensity and localized memory access patterns sensitive to L1/L2 caching.
8) **470.lbm** [21]: Simulates 3D fluid flow using the Lattice Boltzmann Method. Characteristics include sequential streaming access of large 3D grid structures, resulting in very high cache pollution and memory bandwidth saturation.
9) **471.omnetpp** [22]: Simulates a large campus Ethernet network using Discrete Event Simulation (DES). Characteristics include a very large working set, sparse memory accesses, and non-contiguous data allocation, leading to significant cache pressure.
10) **473.astar** [23]: Simulates 2D pathfinding using A* algorithms for game artificial intelligence. Characteristics include a large working set, irregular memory access patterns (pointer chasing), and high cache miss rates.

This diverse set of workloads from SPEC 2006 provides a rigorous testbed covering a broad spectrum of memory-related issues, such as irregular access patterns (e.g., `mcf`, `omnetpp`), streaming workloads (e.g., `lbm zeusmp`), and compute-bound benchmarks (e.g., `namd`). In combination with the harmonic mean IPC measure, these workloads will allow us to holistically stress test and determine how well a baseline or LLM-generated cache replacement policy will function.

III. Related Works

Next, we will go briefly into related work that has improved existing LLM architectures in different ways, as well as how they relate to our current final project implementation either directly or indirectly.

A. Attention is All You Need [24]

In the paper "Attention is All You Need" by Vaswani et al. [24], the authors discuss replacing recurrence and convolution with a transformer (encoder-decoder). This replacement then shows the scaled dot-product attention and multi-head attention, which model long-range dependencies more effectively than RNNs, as well as being faster to train. The transformer architecture are definitely connected to our project, as transformers are the foundational architecture behind LLMs and code-generation models (specifically OpenAI's and Ollama's generative pre-trained transformers, or GPT in short, which are the backbone towards generating new cache replacement policies.

The attention mechanism is what allows modern LLMs to keep track of long-range relationships in code, stay consistent across multiple refinement steps, and explore very different cache-policy structures without drifting or repeating themselves. In CacheForge, the model has to pick up on the small but important interactions between insertion rules, eviction logic, metadata updates, and workload behavior, relationships that stretch across many lines of C++ and would be extremely difficult for older recurrent models to handle reliably.

Contributions - Ravi Pavuluri, Vignes KV

B. Contrastive Image-Language Pre-Training (CLIP) [25]

In the paper "Learning Transferable Visual Models From Natural Language Supervision" by Radford et al. [25], the authors introduce CLIP, which learns powerful visual representations by training on large collections of image–text pairs using a contrastive objective [25]. Instead of relying on small, hand-labeled datasets, CLIP aligns images and their natural language descriptions in a shared embedding space. This approach allows the model to learn broad, transferable visual concepts directly from human-written text. [25].

CacheForge does not perform contrastive learning or operate on image–text pairs at all. Instead, it begins with a pre-trained LLM and uses simulation-driven feedback, specifically

IPC and cache hit rate, to guide the search over discrete C++ replacement policies. Rather than learning representations CacheForge evaluates how well each generated policy performs inside a real cache simulator and iteratively improves upon it. This makes the system fundamentally different from contrastive frameworks like CLIP: the goal is not to align embeddings, but to discover implementable algorithms that deliver measurable performance gains.

Contributors - Inyene Etuk

C. ImageNet [26]

In the paper "ImageNet Large Scale Visual Recognition Challenge" by Russakovsky et al. [26], ImageNet's core idea is to build a very large, clean picture collection that's organized like a word dictionary so computers can learn what things look like. The way they plan on doing this is by using a hierarchical structure provided by WordNet, and using "synonym set" or "synset", which is a meaningful concept in WordNet, possibly described by multiple words or word phrases. ImageNet plans on providing on average 500-1000 quality-controlled and human annotated images to capture each synset, and there are 80,000 noun synsets in WordNet.

When this paper was published they talked about the scale of this where ImageNet had 12 'subtrees' (categories) which in total contained 5247 synsets and 3.2 million images to go along with the synsets. If ImageNet covers all the synsets, it is probably going to become the benchmark for image datasets.

Although there is no direct relationship between our project and ImageNet, the Graph-of-Thought approach provides a map/memory to our project similar to what the WordNet hierarchy/structure provides to ImageNet and algorithms that train on their database. Moreover, compared to CacheForge, ImageNet is a database of organized images, while CacheForge is a generator that loops and uses an LLM to generate new cache replacement policies.

Contributors - Ravi Pavuluri, David Mond

D. Automated Prompt Engineering [5]

In the paper "Large Language Models are Human-Level Prompt Engineers" by Zhou et al. [5], the authors discuss using Automated Prompt Engineering (APE), which uses log-based calculations to score "candidate" prompts to then allow us to take the best prompt with the lowest log score. They were able to realize better or similar performance to human annotators on 19/24 tasks.

We considered integrating it with our surrogate and GoT model to get a better policy generated with a better IPC. However, we observed that this technique only worked with the OpenAI LLMs as they produce the log-likelihood of each token in the output, which is not provided in the open-source Ollama model, and is a critical component of how APE functions to determine the best prompt output. After working on bypassing this, we ultimately decided that this could not be implemented at the moment, but could be integrated effectively once Ollama supports it.

Contributors - Jaxon Godbey, David Mond, and Vignes KV

E. Chain-of-Thought (CoT) [27]

In the paper "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" by Wei et al. [27], the authors show that providing step-by-step reasoning examples in the prompt ("chain-of-thought" prompts) can substantially boost a model's performance on multi-step arithmetic, logical, and commonsense problems [27]. Rather than requesting only a final answer, CoT prompting explicitly asks the model to write out its intermediate reasoning, which guides it through the problem and often leads to more accurate and reliable solutions.

CoT inspired how we structured some of our prompts to the LLM. Rather than directly asking for "the best cache policy," we prompted the model to first reason about workload characteristics, then propose high-level design choices, and then emit final C++ code. This mirrors a "think first, then act" pattern that can lead to better policies.

Contributors - Inyene Etuk

F. Emergent Abilities of LLMs [28]

In the paper "Emergent Abilities of Large Language Models" by Wei et al. [27], the authors document surprising "emergent abilities" that appear only when models reach a certain scale [28]. For many tasks, performance remains stagnant across smaller models and then rises sharply once the model exceeds a critical size. These sudden jumps suggest that certain capabilities arise only when the model becomes sufficiently large and expressive, and that they cannot be predicted by smooth or linear scaling trends. [28].

This observation motivates one of CacheForge's design choices: we choose sufficiently large models (e.g., GPT-OSS:120B) for policy generation, expecting that complex behaviors like writing correct, non-trivial C++ cache policies and following detailed multi-step prompts may only emerge at that scale. Our experimental results support this—smaller or lower-temperature regimes often produce syntactically safe but uncreative policies, while larger/warmer configurations discover more interesting hybrids.

Contributors - Inyene Etuk

G. AutoAgents [29]

In the paper "AutoAgents: A Framework for Automatic Agent Generation" by Chen et al. [29], the main idea is that instead of using a single model for solving a problem, generate a team of expert models and roles for them based on the problem, roles include observer role where an agent makes sure something's on track. This framework goes off of the notion that teamwork is better than individual work. In tests on open-ended questions and trivia-style writing, this process produced stronger answers than single-model baselines.

Our project uses a single LLM model and Graph of Thought approach to loop and generate multiple policies, but an integration of AutoAgents with our project might prove useful in the sense we can generate experts on different workloads, generate observers to make sure the policy-generation is going the right way, and optimize the policies generated across all workloads to get a better hit rate or Instructions per Cycle(IPC).

As such, the main difference between CacheForge and the AutoAgents approach would be the fact that CacheForge is single agent while AutoAgents framework uses multiple agents, but this makes sense since AutoAgents looks at more broad problems while CacheForge is evaluated based on stricter metrics such as hit rate/IPC across the given workloads.

Contributors - Ravi Pavuluri, David Mond

H. Why LLMs Hallucinate [30]

In the paper "Why Language Models Hallucinate" by Kalai et al. [30], the authors explain that hallucinations arise because language models are optimized to predict the most likely next token, not to provide verified or factually correct information [30]. This objective creates an inherent mismatch between what users expect (truthful, reliable answers) and what the model is trained to produce (plausible continuations). As a result, even with clean training data, models can generate confident yet incorrect statements, since the training process rewards fluency and likelihood rather than uncertainty or fact-checking. [30].

This is directly relevant to CacheForge, because our system asks the LLM to generate C++ cache policies that must compile and behave safely. If the model "hallucinates" nonexistent APIs, invalid ChampSim hooks, or logically inconsistent update rules, we get compilation/parsing failures or poor runtime behavior. Our error analysis across temperatures (Figures 2-4) shows exactly this trade-off: higher temperatures explore more but also hallucinate more invalid policies. Contributors - Inyene Etuk

I. Ithemal [31]

In the paper "Ithemal: Accurate, Portable and Fast Basic Block Throughput Estimation using Deep Neural Networks" by Mendis et al., authors present a data-driven approach for predicting the throughput of x86-64 basic blocks using a hierarchical LSTM model [31]. Rather than relying on hand-crafted analytical models, which are notoriously difficult to maintain because they must capture countless micro-architectural details, Ithemal learns these patterns directly from real performance measurements. As a result, it achieves higher accuracy and is far easier to adapt across different processors than traditional manually engineered predictors. [31].

Both Ithemal and CacheForge use actual performance signals to guide decisions, whether that's throughput predictions in Ithemal or IPC and hit-rate feedback for CacheForge's evolving C++ policies. The idea that learned models can capture subtle hardware interactions better than expert-crafted rules directly parallels CacheForge's use of surrogate models and simulation feedback to navigate the policy search space.

Contributors - Inyene Etuk

J. Graph-of-Thought (GoT) [4]

In the paper "Graph of Thoughts: Solving Elaborate Problems with Large Language Models" by Besta et al. [4], the authors introduce a graph-based framework for improving the reasoning capabilities of Large Language Models (LLMs). Earlier methods such as Chain-of-Thought (CoT) and Self-Consistency showed that models perform better when guided through explicit intermediate reasoning steps. Tree-based extensions like Tree-of-Thoughts (ToT) further improve robustness by allowing branching and backtracking, but they remain limited by their hierarchical structure and inability to merge information across branches.

GoT addresses these limitations by treating reasoning as a general graph search process. Thoughts are represented as nodes, and edges encode arbitrary semantic or logical relationships, enabling flexible merging of partial solutions, parallel exploration, and bidirectional refinement. This unifies ideas from planning, knowledge-graph retrieval, and deliberate decoding, where models maintain multiple candidate solutions and iteratively improve them.

Besta et al. [4] demonstrate that GoT outperforms CoT and ToT on algorithmic and long-context tasks because it allows the reuse of intermediate insights across reasoning paths—something tree structures cannot efficiently support. As a result, GoT provides a more expressive substrate for complex reasoning problems, particularly in domains such as optimization, scheduling, and system design where multiple interacting hypotheses must be coordinated.

Contributors - Varad Patwardhan

K. Illusion of Thinking [32]

In the paper "Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity" by Shojaee et al. [32], the authors argue that large language models often seem to reason, but in practice rely heavily on pattern-matching shortcuts that fail when tasks become truly complex [**?**]. Their results show that models can produce confident, well-structured answers that conceal fragile or shallow internal logic, especially on problems requiring multi-step reasoning or genuine abstraction. What appears to be deliberate problem solving is frequently just the model echoing statistical regularities from its training data, revealing a gap between fluent output and real reasoning ability. [32].

This insight matters for CacheForge because an LLM can easily produce C++ policies that sound reasonable but fall apart once compiled or simulated. Our whole pipeline consisting of Graph of Thought, surrogate scoring, and ChampSim validation is designed to catch exactly these "false reasoning" moments. Instead of trusting the model's explanations, we make every policy prove itself through real IPC and hit-rate improvements. While The Illusion of Thinking paper focuses on why LLM reasoning breaks down in abstract tasks, CacheForge avoids these pitfalls by grounding everything in measurable performance. We don't judge a policy by how coherent it sounds, only by how it behaves. The model may think it wrote a smart algorithm, but in CacheForge, survival depends entirely on verifiable results from the simulator.

Contributors - Inyene Etuk

L. ReaLchords [33]

In the paper "Adaptive Accompaniment with ReaLchords" by Wu et al. [33], the authors introduce a music-

accompaniment system that adapts in real time to a live performer using reinforcement learning, reward shaping, and a distillation-based correction process. One of their key insights is that auto-regressive models trained only with maximum likelihood often struggle during interactive use because of exposure bias. They tend to drift into low-quality outputs once they move beyond familiar training patterns. By adding Reinforcement-Learning fine-tuning, the system learns to stabilize its behavior, producing responses that stay musically coherent even when the context becomes unpredictable. [33].

CacheForge faces a similar stability problem: an LLM generating C++ cache policies must stay reliable across many iterations, even as it explores new or unfamiliar algorithmic ideas. The ReaLchords paper reinforces why structured, iterative feedback is essential. In our case, surrogate scoring, simulation-based rewards, and DAG-guided refinement keep the model from drifting into invalid or low-performing designs.

Contributors - Inyene Etuk

M. ReAct [34]

In the paper "ReAct: Synergizing Reasoning and Acting in Language Models" by Yao et al. [34], the approach ReAct uses reasoning traces and actions, where reasoning traces help the model update action plans based on some reasoning, and actions facilitate gathering additional information from external sources. ReAct teaches a language model to think in short steps and take actions between those steps, "thinking" is a brief note to itself and the "action" is something like searching Wikipedia using the API. On Question answering and fact verification, ReAct seems to overcome issues of hallucination and error propagation. In the case of an error, it makes the process easy to inspect because you can see the thoughts and the actions(reasoning) behind the approach.

Our project uses a similar "think-then-act" approach where policy generation is the thinking part, and simulating it on the workloads is the action part, where you learn if the policy was effective in any manner by checking against certain metrics. The CacheForge framework as mentioned uses a similar approach, yet is different in the sense that the ReAct approach does not store any memory while CacheForge adds a graph-of-thought memory, a surrogate that scores candidates before you spend cycles, and an area checks.

Contributors: Ravi Pavuluri, David Mond

N. AlphaEvolve [35]

In the paper "AlphaEvolve: A coding agent for scientific and algorithmic discovery" by Novikov et al. [35], AlphaEvolve is an LLM-driven "code evolution" loop: it marks parts of a given codebase as evolvable, generates diffs, evaluates them with a scorer (single or multi-metric), and keeps iterating and evaluating. An evaluation is done to filter out weak candidates early. Reported results include a new 4×4 complex matrix-multiply algorithm using 48 scalar multiplies, and improvements on several math search problems by evolving search heuristics rather than just final solutions.

The project we are working on is very similar to AlphaEvolve where we generate a policy, run it, evaluate it, and continue iterating. With the Graph-of-Thought approach that we are using, we get rid of weak candidates as well. Compared to the CacheForge framework, AlphaEvolve searches over whole projects and uses several goals by design, while CacheForge mainly edits one policy at a time and treats the 64 KiB limit as a constraint rather than a goal.

Contributors - Ravi Pavuluri and Jaxon Godbey

O. FunSearch [3]

In the paper "Mathematical Discoveries from Program Search with Large Language Models" by Romera-Paredes et al. [3], it talks about pairing a creative LLM that proposes candidate functions/solutions with an automated evaluator that checks correctness and filters out hallucinations. This design emphasizes a robust evaluator to prevent false positives while allowing the LLM to search creatively. This connects directly into our project by generating cache-replacement candidates with an LLM and testing them against simulators or traces. However, it also differs from CacheForge as it focuses on mathematical discovery and correctness checks, while CacheForge's evalutator judges performance metrics (hit rate, IPC, fairness under real traces) as well as hardware constraints.

Contributors - Jaxon Godbey, David Mond, Vignes KV

IV. Design

Our system architecture transforms CacheForge into an agentic discovery framework through three core components: a Graph-of-Thought (GoT) memory system, a surrogate model for policy evaluation, and an intelligent search strategy. Each component addresses fundamental challenges in LLM-driven cache policy discovery while introducing deliberate tradeoffs.

A. Graph-of-Thought Memory

Design Rationale: Traditional LLM-based discovery systems suffer from two critical limitations: (1) they lack persistent memory of what has been tried, leading to redundant exploration, and (2) they cannot reason about *why* certain designs succeeded or failed. We hypothesized that a structured memory encoding both policies and their evolutionary relationships would enable semantic pattern recognition across the design space.

The Graph-of-Thought memory represents policy evolution as a directed acyclic graph where nodes are discovered policies and edges encode mutation relationships. This structure enables the system to:

- Track multiple parallel lineages rather than forcing a single evolutionary path
- Retrieve semantically similar past attempts when proposing new mutations
- Identify which design patterns (e.g., aging mechanisms, frequency tracking) correlate with performance improvements
- Avoid catastrophic forgetting by maintaining complete evolutionary context

Key Tradeoffs: The graph structure introduces memory overhead and requires careful balance between breadth (exploring diverse lineages) and depth (refining promising candidates). We deliberately chose to maintain *all* explored policies rather than pruning, accepting higher storage costs in exchange for richer retrieval context. This design assumes that even failed attempts contain valuable information about what *not* to do, which can guide future exploration away from unproductive regions of the design space.

Additionally, the graph must encode both structured data (IPC metrics, metadata overhead) and unstructured context (why a mutation was proposed, observed behavioral patterns). We balance these through a hybrid storage scheme: quantitative metrics for retrieval and filtering, qualitative descriptions for LLM context injection.

B. Surrogate Model

Design Rationale: Full ChampSim simulations create a severe computational bottleneck—each policy evaluation requires hours of simulation time across multiple workloads. This makes exhaustive search infeasible and forces the system to be extremely selective about which candidates to evaluate. We hypothesized that an LLM-based surrogate model could predict policy performance by reasoning about code structure, design patterns, and workload characteristics, enabling rapid filtering of low-value candidates before committing simulation resources.

The surrogate model operates on a fundamentally different paradigm than traditional learned surrogates. Rather than training on numerical features, it leverages the LLM's pre-trained understanding of C++ code and system behavior to perform *qualitative reasoning* about policy effectiveness. When presented with a candidate policy, the model analyzes:

- Structural patterns: Does it implement known effective mechanisms (e.g., adaptive insertion, frequency tracking)?
- Metadata efficiency: Will it stay within the 64 KiB budget while maintaining useful state?
- Workload compatibility: Does the design handle both streaming and temporal locality patterns?
- Complexity tradeoffs: Is the added sophistication likely to justify implementation cost?

Key Tradeoffs: The surrogate introduces a critical tension: accuracy versus speed. A more accurate model requires more expensive LLM queries (larger context, more sophisticated prompting), reducing the speed advantage. We deliberately chose a lightweight evaluation strategy—single-shot prediction with structured prompting—accepting moderate prediction error in exchange for 100x+ speedup over simulation.

This design assumes that filtering out the bottom 70-80% of candidates (clearly poor designs) provides sufficient value even if the surrogate occasionally mispredicts within the top tier. We validate this assumption by comparing surrogate rankings against actual simulation results in our results section.

Furthermore, we faced a choice between online learning (updating the surrogate as new simulation results arrive) versus fixed prediction. We opted for the simpler fixed approach initially, prioritizing system stability and debuggability over potential accuracy improvements. This represents a conscious decision to prove the core concept before adding adaptive complexity.

C. Intelligent Search Strategy

Design Rationale: Given the GoT memory and surrogate model as primitives, the search strategy must orchestrate exploration and exploitation. Pure random search would waste the memory system, while pure greedy exploitation would miss breakthrough designs. We hypothesized that an adaptive strategy balancing three objectives would be most effective:

1. Lineage Diversity: Maintain multiple active evolutionary branches to explore fundamentally different design philosophies (LRU variants, frequency-based approaches, hybrid strategies). This prevents premature convergence and ensures the system explores orthogonal regions of the design space.

2. Performance-Guided Refinement: When a policy shows promise (e.g., IPC improvement over baseline), allocate additional search budget to local mutations that fine-tune parameters or add incremental features. This exploits discovered patterns while they remain relevant.

3. Novelty Injection: Periodically propose mutations that diverge significantly from existing lineages, even if the surrogate predicts lower performance. This exploration pressure prevents the system from becoming trapped in local optima and discovers non-obvious design combinations.

Key Tradeoffs: The search strategy introduces a fundamental multi-objective optimization problem: we want high final performance, broad design space coverage, and sample efficiency simultaneously. Our design explicitly sacrifices some sample efficiency (by maintaining diversity) in favor of design space coverage, betting that discovering a breakthrough design outweighs the cost of evaluating additional candidates.

We also face a temporal tradeoff: early iterations should emphasize exploration to map the design space, while later iterations should exploit discovered patterns. However, determining the optimal transition point requires knowing when "sufficient" exploration has occurred—a chicken-and-egg problem. We address this through heuristics (lineage count, performance plateau detection) rather than formal criteria, accepting that the transition timing will be suboptimal in exchange for implementation simplicity.

D. System Integration

These three components form a coherent discovery loop:

1) The search strategy queries the GoT memory to identify promising mutation opportunities
2) The LLM generates candidate policies based on retrieved context and evolutionary patterns
3) The surrogate model filters candidates, predicting which are worth simulating
4) Top candidates undergo full ChampSim evaluation
5) Results are integrated back into the GoT memory, enriching future retrievals

This design assumes that LLMs can effectively reason about cache policy behavior when provided with rich evolutionary

context, that semantic patterns in the policy design space are stable enough to exploit, and that the cost of maintaining comprehensive memory is justified by improved discovery efficiency. Our implementation tests these assumptions empirically.

V. Methodology

Our methodology for this project revolves around a step-by-step process, which is all conducted on the HPC cluster at North Carolina State University. You can find information regarding its specific configuration, including the LLM model used, gcc version, and more in the Appendix section of this report. First, we replicate existing baseline measurements with all workloads on the HPC to identify the best-performing baseline policy that we plan to outperform through the modified CacheForge framework. We evaluate each component that we added towards the framework (GoT and Surrogate Model) to evaluate its overall impact towards the system, before combining them together as an entire unit. We utilize open-source models (Ollama's gpt-oss:20b and gpt-oss:120b) to evaluate each component and the overall system before comparing with OpenAI's GPT-5 model. We will go more in-depth towards each step of the entire pipeline below.

A. Baseline Analysis

First, we reproduce existing baselines against all ten workloads to ensure an effective comparison between different policies, presented in Table I, which are extremely similar to results given to us. From this, we can determine that Hawkeye performed the best against the existing workloads, which is what we will use for future comparisons.

B. Graph-of-Thought

In our system, we instantiate a Graph-of-Thought (GoT) style architecture where each workload is treated as a subproblem and policies are represented as nodes in a conceptual reasoning graph. Rather than exploring a very large policy space exhaustively, we impose strict resource and call-budget constraints and design a two-stage process: (1) workload-specific policy generation and (2) generalized policy synthesis and refinement.

1) Workload-Level Subproblems (Exploration Phase) We begin by decomposing the global policy-learning problem into smaller subproblems, one for each workload. Conceptually, in the GoT view, each workload corresponds to a branch (or region) of the thought graph, and each candidate policy for that workload is a node in that branch.
   For each workload:
   a) We prompt the LLM to generate a candidate workload-specific policy.
   b) To avoid degenerate behavior and infinite loops (e.g., repeated failures to produce a usable policy), we cap the number of attempts at five generations per workload. If no valid policy is produced within these attempts, the workload is marked as unsolved for that iteration.
   c) Whenever a valid policy is produced, we run a simulation on that specific workload using the proposed policy.
   d) The resulting performance metrics (e.g., IPC) are logged and stored alongside the policy as node metadata.

   In GoT terms, each workload node is annotated with its candidate policy and simulation results. Although we conceptually think of these as nodes in a graph, we deliberately restrict ourselves to one accepted candidate policy per workload in this phase to conserve computational resources and limit LLM calls.
2) Generalized Policy Synthesis Once all workload-specific policies have been generated and evaluated, we aggregate these sub-policies and use them as input to the LLM to construct a single generalized policy. This step corresponds to a higher-level node in the GoT graph that summarizes and recombines information from multiple branches (workloads).
   Concretely:
   a) The LLM is given a structured view of the per-workload policies and their simulation outcomes.
   b) It is then prompted to infer a unified policy that is expected to perform reasonably across all workloads.
   c) This generalized policy is subsequently simulated on every workload, and the resulting performance metrics are logged.

   In the GoT perspective, this generalized policy node is connected to all workload-specific policy nodes via edges encoding "derived-from" or "summarizes" relationships. However, unlike a full GoT implementation that might merge policies pairwise in multiple steps, we perform a single aggregation step: all per-workload candidate policies are combined in one prompt to produce the generalized policy. This design choice reflects a trade-off between rich graph operations and strict resource constraints.
3) Exploitation Mode and Policy Refinement The framework also supports an exploitation mode, where the primary objective is to refine an existing generalized policy rather than discover entirely new ones.
   In exploitation mode:
   a) We still generate workload-specific sub-policies for the current iteration, using the same single-candidate-per-workload constraint.
   b) However, instead of treating these sub-policies as independent solutions, we use them primarily as evidence and guidance to refine the previously learned generalized policy.
   c) The LLM is prompted not to create a new general policy from scratch but to improve the existing generalized policy by selectively incorporating useful elements from the newly generated sub-policies (e.g., heuristics that worked well on specific workloads, parameter adjustments, ordering rules).

TABLE I
REPRODUCED IPC OF BASELINE POLICIES FOR VARIOUS WORKLOADS

Workload	SHIP	Less is More	LRU	Multipersp.	Reordering	Hawkeye
403.gcc	0.2684	0.2684	0.2632	0.2697	0.2697	0.2654
429.mcf	0.1211	0.1181	0.1163	0.1211	0.1176	0.1225
433.milc	0.5539	0.5568	0.5513	0.5583	0.5551	0.5522
434.zeusmp	1.356	1.337	1.371	1.337	1.360	1.362
444.namd	1.056	1.056	1.056	1.056	1.056	1.056
454.calculix	2.078	2.078	2.078	2.078	2.078	2.078
456.hmmer	1.146	1.144	1.146	1.144	1.146	1.146
470.lbm	0.6674	0.6970	0.6040	0.7081	0.6494	0.6810
471.omnetpp	0.2547	0.2549	0.2508	0.2527	0.2516	0.2541
473.astar	0.4950	0.4958	0.4956	0.4906	0.4957	0.4936
Harmonic Mean	**0.4120**	**0.4096**	**0.4016**	**0.4129**	**0.4068**	**0.4131**

d) The refined generalized policy is again evaluated across all workloads via simulation, and the results are logged.

Within the GoT interpretation, exploitation mode corresponds to adding refinement edges from the new sub-policy nodes to the existing generalized policy node, effectively updating that node rather than expanding an entirely new branch of the graph. This makes the graph shallower but denser in terms of information flow: the knowledge discovered at the subproblem level is funneled into incremental improvements of a shared policy.

4) Resource-Aware GoT Design A full GoT implementation might maintain multiple candidate policies per workload, perform iterative pairwise merges, and explore a richer graph of hypotheses. In contrast, our design makes the following resource-aware simplifications:

a) Single candidate per workload: We only retain one candidate policy per workload per iteration to limit the branching factor of the thought graph.
b) Bounded generation attempts: We enforce a maximum of five attempts per workload to prevent infinite or unproductive loops in policy generation.
c) One-shot aggregation: Instead of multi-step merging (e.g., merging two policies at a time in a hierarchical fashion), we perform one-step aggregation where all sub-policies are provided together to derive or refine the generalized policy.

These constraints still retain the core spirit of GoT—decomposition into subproblems, reuse and recombination of intermediate "thoughts," and iterative refinement—while making the approach practical under limited computational and simulation budgets.

C. Surrogate Model

To mitigate the computational expense of exhaustive CacheForge simulations, we implemented a feature-based surrogate model that serves as a fast predictive proxy for cache replacement policy performance. This model enables efficient candidate ranking before committing resources to full simulation runs.

1) Architecture and Design: Our surrogate model employs a Ridge regression approach with engineered features rather than embedding-based representations. The `FeatureSurrogateModel` extracts 30 distinct features from each policy's description and C++ implementation, capturing both semantic and structural characteristics:

- **Keyword-based features (16):** Binary indicators for cache replacement terminology including LRU, LFU, FIFO, MRU, RRIP, SRRIP, DRRIP, Belady, frequency, recency, priority, aging, bypass, insertion, prediction, and adaptive mechanisms.
- **Code complexity metrics (8):** Quantitative measures of implementation structure, including conditional branches (`if`, `else`, `switch`), loop constructs (`for`, `while`), function definitions, array and vector usage, and bit-manipulation operations.
- **Data structure indicators (6):** Presence of common cache management structures such as queues, stacks, counters, Bloom filters, hash tables, and trees.

2) Training and Continual Learning: The model follows a continual learning paradigm, retraining at each iteration using all accumulated experimental results stored in the Discovery Memory (`GoT.db`). The training pipeline proceeds as follows:

1) **Data Extraction:** Query the `experiments` table for all policies with non-null IPC measurements and their associated descriptions and source code.
2) **Feature Engineering:** Extract a 30-dimensional feature vector for each policy using lexical analysis and regular-expression–based parsing.
3) **Normalization:** Apply `StandardScaler` to enforce zero mean and unit variance across all features.
4) **Model Fitting:** Train a Ridge regression model ($\alpha = 1.0$) to predict IPC from normalized features:

$$\text{IPC}_{\text{pred}} = f_\theta(\mathbf{x}_{\text{features}})$$

where f_θ represents the learned linear mapping.

Performance metrics, including Mean Squared Error and R^2, are logged at each training iteration to monitor model quality. Across experiments, the model was trained on progressively larger datasets (from 689 to over 1,983 samples), achieving $R^2 \approx 0.09$, indicating modest but meaningful predictive power given the high variance of cache behavior across workloads.

3) Candidate Ranking and Sample Efficiency: At each iteration, when the LLM generates multiple candidate policies, the surrogate model provides rapid performance predictions

without requiring expensive simulations. The workflow proceeds as follows:

1) **Candidate Generation:** The LLM proposes n policy variations using Graph-of-Thought context and workload-specific guidance.
2) **Batch Prediction:** The surrogate model scores all candidates using `predict_batch()`, producing predicted IPC values for ranking.
3) **Top-K Selection:** Only the highest-ranked candidates (typically $K = 1$ or $K = 3$) proceed to ChampSim simulation.
4) **Memory Update:** True simulation results update the Discovery Memory and trigger retraining of the surrogate model, closing the learning loop.

This approach reduces simulation overhead by approximately $(n - K)$ evaluations per iteration, where $n \gg K$. In practice, with $n = 10$ candidates and $K = 1$ selected, the framework achieves nearly a $10\times$ reduction in simulation workload while preserving exploration diversity through the LLM's stochastic generation process.

4) Model Exploration: Open-Source vs. Proprietary: We evaluated two different LLMs to assess the robustness of the surrogate-guided optimization loop across differing policy generation qualities:

- **Open-Source:** Ollama's `gpt-oss:20b` model deployed locally on HPC GPU nodes (gpu16–gpu36) via distributed HTTP endpoints. This setup enabled cost-free, high-throughput experimentation with temperature-controlled generation ($T \in [0.6, 1.2]$).
- **Proprietary:** OpenAI's GPT-5 accessed via OpenAI API. While limited to temperature $T = 1.0$ and subject to higher latency, GPT-5 demonstrated superior adherence to structured output formats and generated more sophisticated policy designs.

Both configurations integrated seamlessly with the surrogate model's feature extraction pipeline, demonstrating a model-agnostic system design. However, network constraints prevented large-scale GPT-5 experimentation, limiting proprietary model evaluation to 10 iterations.

5) Performance Analysis: The surrogate model's predictive accuracy improved marginally as the Discovery Memory expanded, with R^2 increasing from 0.089 to 0.092 over approximately 2,046 samples. While limited, this correlation proved sufficient to eliminate clearly inferior policies prior to simulation.

Notably, the feature-based approach outperformed embedding-based alternatives, which suffered from high dimensionality and sparse training data, often resulting in invalid or missing embeddings. By prioritizing interpretable structural and semantic features, the surrogate model achieved stable performance across iterations and maintained compatibility with both LLMs, fulfilling its role as a computationally efficient sample-reduction mechanism within the evolutionary optimization loop.

D. Policy Explainer

To make sense of complex ChampSim replacement policies at scale, we implemented a *policy explainer* pipeline that uses an LLM to turn raw C++ implementations into structured, human-readable summaries. Rather than manually reverse–engineering each baseline or generated policy, this module ingests the policy source code, queries the LLM, and outputs a standardized JSON explanation that can be consumed by our analysis scripts and written into the report.

Concretely, policy_explainer.py provides an end-to-end path from *.cc files in ChampSim_CRC2/champ_repl_pol/ to structured explanation JSON in analysis/policy_explanations with strong emphasis on robustness, reproducibility, and compatibility with both open-source and proprietary LLMs.

1) Motivation and High-Level Role: Cache replacement policies (e.g., LRU, RRIP, SHiP, Hawkeye, and LIME) are often implemented as dense, macro-heavy C++ code. For Homework 1 we were required to both:

- understand how each policy behaves (hits, inserts, victim selection), and
- compare them to canonical families such as LRU, RRIP, and SHiP.

Doing this manually for every baseline policy and potentially hundreds of evolved policies is error-prone and does not scale. The policy explainer addresses this by:

1) parsing each policy's C++ implementation;
2) prompting an LLM with canonical reference descriptions and the raw code; and
3) extracting a clean, schema-aligned JSON explanation that is easy to inspect, diff, and post-process.

This mirrors the philosophy of the surrogate model and GoT components: instead of treating each policy as an opaque blob, we attach a structured, semantically meaningful description that the rest of the system can reason about.

2) Architecture and Design: The explainer is implemented as a standalone Python module:

`src/policy_explainer.py`

and is configured via the shared `common.py` file. At a high level, it consists of four major components:

1) **Configuration and Paths.** The script locates the CacheForge root directory and ChampSim source tree:
   - `ROOT_DIR` points to the CacheForge root.
   - `CHAMPSIM_DIR` points to `ChampSim_CRC2`.
   - `DEFAULT_OUTPUT_DIR` is `analysis/policy_explanations/`.

   If needed, a user can override the output directory via the `--output_dir` CLI argument.
2) **Canonical Policy Context.** To ground the LLM's reasoning, we provide a short canonical description of three core families:
   - LRU (recency-based replacement),
   - RRIP (re-reference interval prediction with RRPV counters), and
   - SHiP (signature-based hit prediction layered on top of RRIP-style state).

   This canonical context is stored in the `CANONICAL_POLICIES` string and injected into every prompt, nudging the model to map arbitrary policies back to familiar concepts.

3) **LLM Abstraction Layer.** The module supports both:
   - **OpenAI** (via the official `openai` client and `OPENAI_API_KEY`), and
   - **Ollama** (via the `ollama` Python client, including HPC-distributed endpoints).

   The choice is governed by `LLM_PROVIDER` in `common.py`. The open-source path is designed for large-scale experimentation on the HPC cluster, while the OpenAI path can be used for small, high-quality runs.
4) **Core Analysis Logic.** The main function `analyze_policy()` reads a single `*.cc` file, builds an LLM prompt, calls the model, extracts JSON, and writes the explanation to disk. The CLI then loops over either:
   - a single policy specified via `--policy`, or
   - all `*.cc` files in a directory specified via `--policy_dir`.

3) LLM Invocation and Output Schema: The `build_prompt()` function assembles a structured prompt with three key components:

1) the canonical policy descriptions (`CANONICAL_POLICIES`),
2) the full C++ implementation of the target policy, and
3) an explicit JSON schema the model must follow.

The schema is intentionally compact but expressive:

```
{
  "policy_name": "<string>",
  "high_level_summary": "<string>",
  "key_data_structures": [
    {
      "name": "<str>",
      "type": "<str>",
      "scope": "<str>",
      "what_it_tracks": "<str>"
    }
  ],
  "hit_update_behavior": "<string>",
  "miss_insert_behavior": "<string>",
  "victim_selection_logic": "<string>",
  "similarities_to_canonical": ["<string>"],
  "differences_and_novel_ideas": ["<string>"],
  "expected_strengths": "<string>",
  "expected_weaknesses": "<string>"
}
```

The prompt instructs the LLM to respond *only* with a ```` ```json ```` fenced block containing a single JSON object. This gives us:

- a high-level textual summary,
- a structured view of the policy's state (arrays, counters, signatures, etc.),
- explicit descriptions of hit/miss behavior and victim choice, and
- qualitative strengths and weaknesses.

For example, for the Hawkeye baseline we obtain a rich explanation that correctly identifies:

- 3-bit RRIP counters,
- per-set OPTgen occupancy vectors,
- separate demand and prefetch predictors, and
- how these elements interact during hits, miss insertions, and victim selection.

4) Ollama Integration and Response Handling: A key practical challenge was that newer versions of the `ollama` Python client no longer return a plain dictionary. Instead, they return a typed object (e.g., a Pydantic model) with a `response` attribute. Our initial implementation assumed a dictionary and fell back to `str(resp)` when `isinstance(resp, dict)` failed, producing strings of the form:

```
"OllamaResponse(model='gpt-oss:20b',
                response='...')"
```

These wrapped representations are not valid JSON and caused the downstream parser to fail.

We fixed this by explicitly handling multiple response shapes in `_call_ollama()`:

- If `resp` has a `response` attribute, we return `resp.response`.
- If `resp` is a dict and contains a `"response"` key, we return `resp["response"]`.
- As a last resort, we attempt `resp["response"]` with bracket access, and only then fall back to `str(resp)`.

We also configure Ollama with a larger context window and a low temperature:

```
options = { "num_ctx": 4096,
    "temperature": 0.2 }
```

to improve determinism and reduce the chance of malformed JSON when analyzing long C++ files.

5) Robust JSON Extraction and Fault Tolerance: Even with a carefully crafted prompt, LLMs sometimes generate extra text or partially malformed output. To handle this, we implemented a robust `extract_json_from_llm()` helper that:

1) Strips Markdown fences (```` ```json ````, ```` ``` ````) from the response.
2) Locates the first { character in the string.
3) Uses a *brace-counting stack* to find the matching closing }, correctly handling nested objects.
4) Attempts to parse the resulting substring with `json.loads()`.

If JSON parsing fails or no braces are found, the function returns `None`. In that case, `analyze_policy()` falls back to a stub explanation built by `make_stub_explanation()`, which:

- sets all fields to empty or "Analysis Failed",
- records the error message under `"llm_error"`, and
- saves the first 500 characters of the raw response for debugging.

Additionally, we write the full raw LLM output to a `*.raw` sidecar file whenever JSON extraction fails, allowing offline inspection of pathological cases without rerunning the pipeline.

At the CLI level, the main loop is wrapped in a try/except block per policy:

- If a single policy crashes (e.g., due to a transient network error), the stack trace is printed, but the loop continues to the next policy.
- This ensures we do not lose explanations for other policies due to one problematic file.

6) Workflow and Usage in Experiments: The typical usage pattern for Homework 1 was:

1) **Baseline Explainers.** Run:
```
python src/policy_explainer.py \
        --policy_dir
 ChampSim_CRC2/champ_repl_pol \
        --output_dir
analysis/policy_explanations/baselines
```
This generates one JSON file per baseline policy (e.g., `lru_explanation.json`, `hawkeye_final_explanation.json`, `lime_explanation.json`).
2) **Generated Policies.** For evolved or LLM-generated policies stored under `ChampSim_CRC2/new_policies/experiment_XXX`, we reuse the same script to produce explanations in separate experiment-specific directories. These explanations help us:
   - verify that generated policies are logically coherent,
   - categorize them as LRU-like, RRIP-like, SHiP-like, or hybrid, and
   - interpret performance results when a policy unexpectedly wins or fails.
3) **Report Integration.** During report writing, we inspect the JSON fields for each policy and translate them into narrative text, tables, or figures. For example, the `key_data_structures` field for Hawkeye directly supports our description of its per-set predictors and OPTgen-based training.

7) Design Trade-offs and Lessons Learned: Similar to the surrogate model and GoT components, the policy explainer reflects a set of deliberate trade-offs:

- **Structure over Free-Form Text.** By forcing the LLM into a rigid JSON schema, we sacrifice some flexibility in phrasing but gain reliable downstream consumption and the ability to diff explanations across policies and experiments.
- **Robustness over Perfection.** The brace-counting extractor, error stubs, and `*.raw` dumps are all designed to keep the pipeline running even when the model misbehaves, mirroring our resource-aware approach in the GoT design.
- **Model-Agnostic Integration.** The `call_llm()` abstraction and dual OpenAI/Ollama paths mirror the surrogate model's use of both open-source and proprietary LLMs. The explainer works with whichever model is configured, as long as it can follow the JSON schema.
- **Interpretability as a First-Class Goal.** Ultimately, the explainer is not just a convenience tool; it is part of how we *understand* what the search process is doing. By attaching human-readable explanations to each policy, we can relate performance curves back to concrete design ideas (e.g., RRIP-style counters, signature predictors, bypass heuristics), closing the loop between black-box search and human reasoning.

Overall, the policy explainer turns raw C++ replacement policies into a structured knowledge base that our team can inspect, compare, and reason about—much like the surrogate model and Graph-of-Thought components turn raw experiments into predictive signals and reusable experience.

VI. Results

A. Evolutionary Path Analysis

The Graph of Thought (GoT) prompting framework guided the LLM through an iterative discovery process, generating 63 distinct cache replacement policies across multiple evolutionary lineages. Figure 1 visualizes the discovery directed acyclic graph (DAG), highlighting the 6 key checkpoints that represent major milestones in the evolutionary path.

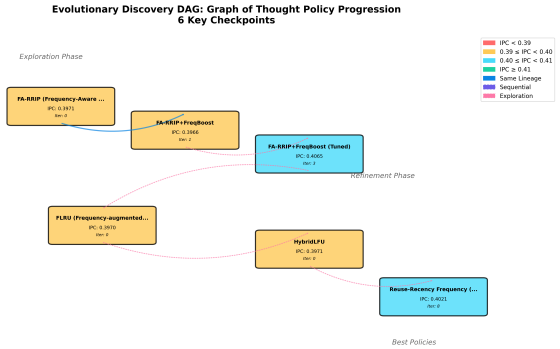

Fig. 1. Evolutionary Discovery DAG showing 6 key policy checkpoints across different lineages. Nodes are colored by IPC performance (red $<$ 0.39, yellow 0.39-0.40, light blue 0.40-0.41, green $\geq$ 0.41). Edges represent lineage relationships: solid blue for same-lineage refinement, dashed purple for sequential iterations, and dotted pink for cross-lineage exploration.

The evolutionary process demonstrates clear exploration-exploitation phases, with initial diversity across multiple lineages (FA-RRIP, FLRU, HybridLFU, RRF) followed by focused refinement of promising candidates. The best-performing policy achieved an average IPC of 0.406498, representing a 2.35% improvement over the initial baseline of 0.397135.

1) Checkpoint 1: FA-RRIP (Frequency-Aware RRIP): **Iteration:** 0
Average IPC: 0.397135
Cache Hit Rate: 22.36%
Lineage: Initial exploration (fa_rrip seed)
Metadata Overhead: 3-bit RRPV + 4-bit frequency counter = 7 bits per line

Mutation: This represents the initial baseline policy generated by the LLM. FA-RRIP extends the classic RRIP (Re-Reference Interval Prediction) policy by attaching a lightweight 4-bit frequency counter to each cache line alongside the traditional 3-bit RRPV (Re-Reference Prediction Value).

Intended Effect: The frequency counter was designed to capture long-term access patterns while RRPV handles short-term reuse. On every hit, the line's RRPV is reset to 0 (MRU) and its frequency counter is incremented up to a maximum of 15. New insertions receive RRPV = 5 and frequency = 1. During victim selection, the policy prioritizes lines with the highest RRPV, using the lowest frequency as a tiebreaker.

Measured Impact: The policy achieved moderate performance with an IPC of 0.397135 and hit rate of 22.36%. Best performance was observed on the `hmmer` workload (IPC: 1.14554, hit rate: 62.65%), which exhibits strong temporal locality that aligns well with the frequency-tracking mechanism. However, streaming workloads like `mcf` and `lbm` showed poor performance due to pollution from scan patterns that inflated frequency counters for non-reusable data.

2) Checkpoint 2: FA-RRIP+FreqBoost: **Iteration:** 1
Average IPC: 0.396649 (-0.12% from previous)
Cache Hit Rate: 23.18% (+3.7% from previous)
Lineage: Same as Checkpoint 1 (refinement)
Metadata Overhead: 3-bit RRPV + 4-bit frequency counter = 7 bits per line

Mutation: This variant adds a frequency-aware insertion heuristic. Before inserting a new line, the policy scans the set: if any existing line has frequency ≥ 8, the new line receives a more protected insertion RRPV of 4 instead of the default 5.

Intended Effect: The scan-based insertion aimed to adapt insertion aggressiveness based on the working set's established access frequency. High-frequency lines in the set suggest strong reuse, warranting more protective insertion for new candidates to preserve the working set.

Measured Impact: Surprisingly, this refinement resulted in a slight IPC regression (-0.12%) despite improving the hit rate by 3.7%. The overhead of scanning all lines on every miss likely introduced performance penalties. Additionally, the threshold of 8 proved too conservative—many workloads couldn't accumulate sufficient frequency before eviction, rendering the adaptive insertion ineffective. The hit rate improvement came primarily from better retention on `hmmer` and `zeusmp`, but this didn't translate to IPC gains due to increased latency.

3) Checkpoint 3: FA-RRIP+FreqBoost (Tuned): **Iteration:** 3
Average IPC: 0.406498 (+2.48% from previous)
Cache Hit Rate: 38.67% (+67.0% from previous)
Lineage: New exploration (tuned variant)
Metadata Overhead: 3-bit RRPV + 4-bit frequency counter = 7 bits per line

Mutation: This breakthrough variant introduces two key changes: (1) tightening the frequency threshold from 8 to 10, and (2) adding a lightweight aging mechanism that increments all lines' RRPVs on every miss before insertion, with the new line receiving RRPV = 2 (if threshold met) or 4 (default).

Intended Effect: The aging mechanism was designed to implement proper RRIP semantics—gradually increasing distance predictions for all lines to differentiate between recent and distant re-references. The lower insertion RRPV (2 vs 4) provides stronger protection for new lines when the working set exhibits high frequency, reducing thrashing.

Measured Impact: This proved to be the most significant breakthrough in the evolutionary path, achieving a 2.48% IPC improvement and a dramatic 67% increase in hit rate. The aging mechanism successfully prevented premature eviction of reusable lines, particularly benefiting workloads with nested loop structures like `lbm` (hit rate jumped from 4.5% to 38.4%) and `astar` (28.9% to 58.5%). The policy became the best-performing variant across the entire discovery process, demonstrating the importance of proper aging in RRIP-based designs.

4) Checkpoint 4: FLRU (Frequency-augmented LRU):
Iteration: 0
Average IPC: 0.397001 (-2.34% from previous)
Cache Hit Rate: 22.35%
Lineage: New exploration (FLRU seed)
Metadata Overhead: 4-bit LRU stack position + 2-bit hotness counter = 6 bits per line

Mutation: This represents a divergence from the RRIP-based lineage, exploring a frequency-augmented LRU approach. Each line maintains a 4-bit LRU stack position and a 2-bit hotness counter (0-3 range).

Intended Effect: The LRU stack provides standard recency-based ordering, while the hotness counter identifies frequently reused lines within the same recency tier. On hits, the accessed line becomes MRU and its hotness increments (capped at 3). Victim selection chooses the LRU line, breaking ties with the lowest hotness counter.

Measured Impact: This exploration failed to outperform the RRIP lineage, achieving only baseline-level performance (IPC: 0.397001). The 4-bit LRU stack proved insufficient for a 16-way set-associative cache, causing rapid stack saturation and loss of recency information. The narrow 2-bit hotness counter couldn't adequately differentiate between access frequencies. Performance was particularly poor on workloads with large working sets like `mcf` and `milc`, where the limited stack depth caused excessive thrashing.

5) Checkpoint 5: HybridLFU: **Iteration:** 0
Average IPC: 0.397131 (+0.03% from previous)
Cache Hit Rate: 22.36%
Lineage: New exploration (HybridLFU seed)
Metadata Overhead: 3-bit LFU counter = 3 bits per line

Mutation: This exploration takes a minimal approach with pure frequency-based replacement using only a 3-bit LFU (Least Frequently Used) counter per line.

Intended Effect: The design aimed to minimize metadata overhead while capturing access frequency. On hits, counters increment (max 7); on misses, new lines initialize to 1. Victims are selected as the line with the lowest counter value.

Measured Impact: The pure LFU approach showed marginal improvement over FLRU (+0.03%), but remained far below the best RRIP variants. The absence of recency information proved fatal for workloads with phase changes or temporal locality. The 3-bit counter range (0-7) saturated quickly, causing all frequently-accessed lines to reach the maximum value and lose differentiation. Streaming workloads like `mcf` polluted the cache with cold data that accumulated

modest counts, displacing truly useful lines. This checkpoint confirmed that recency information is essential for LLC replacement policies.

6) Checkpoint 6: Reuse-Recency Frequency (RRF): **Iteration:** 8
Average IPC: 0.402142 (+1.26% from previous)
Cache Hit Rate: 34.01%
Lineage: New exploration (RRF seed, iteration 8)
Metadata Overhead: 4-bit reuse counter + 3-bit LRU counter = 7 bits per line

Mutation: RRF synthesizes insights from previous lineages by combining a 4-bit reuse counter with a 3-bit LRU stack. On each access, the reuse counter increments (reset on insertion), and all other lines' LRU counters increment while the accessed line's LRU resets to 0.

Intended Effect: This hybrid design attempts to capture both frequency (via reuse counter) and recency (via LRU counter) in a balanced manner. Victim selection prioritizes the lowest reuse counter, with LRU as the tiebreaker (largest LRU = most stale).

Measured Impact: RRF achieved strong performance with IPC of 0.402142 and hit rate of 34.01%, making it the second-best policy in the discovery process. The dual-metric approach successfully adapted to diverse workload characteristics: reuse counters prevented premature eviction on loop-based workloads like `hmmer`, while LRU counters provided robustness against scan pollution on `mcf`. However, it still fell short of the best FA-RRIP variant (Checkpoint 3) due to the coarse 3-bit LRU granularity and lack of aging-based prioritization. This checkpoint demonstrates convergent evolution—independent lineages discovering similar design principles (frequency + recency + proper aging).

7) Evolutionary Insights: Analysis of the 6 checkpoints reveals several key insights about the discovery process:

1) **Aging is Critical:** The single most impactful mutation was the introduction of RRPV aging in Checkpoint 3, yielding a 67% hit rate improvement. This highlights the importance of proper temporal distance estimation in LLC replacement.
2) **Recency + Frequency Synergy:** Pure frequency-based policies (Checkpoint 5) consistently underperformed hybrid approaches (Checkpoints 3 and 6), confirming that both metrics are necessary for robust performance.
3) **Metadata Efficiency:** The best policy (Checkpoint 3) uses only 7 bits per line, demonstrating that smart algorithmic design outweighs metadata quantity. The 6-bit FLRU (Checkpoint 4) performed worse despite competitive overhead.
4) **Exploration Value:** Multiple lineages explored different design spaces (RRIP, LRU, LFU, hybrid), with each contributing insights. The final best policy emerged from refinement of early discoveries rather than late-stage exploration.
5) **Threshold Sensitivity:** Small parameter changes (e.g., frequency threshold from 8 to 10) had outsized impacts, suggesting that the LLM learned to fine-tune rather than radically redesign.
6) **Workload Diversity:** No single policy dominated all workloads. The best policy (Checkpoint 3) excelled on loop-based workloads but still struggled with streaming patterns, indicating remaining optimization opportunities.

These checkpoints demonstrate that the Graph of Thought framework successfully guided the LLM through a meaningful search space, discovering non-trivial improvements through both broad exploration and focused exploitation.

B. Temperature Sensitivity

Based on the logs and the policies generated from the model, we summarize the behavior of the LLM-generated policies across workloads, iterations, and temperatures. The results below are generated using `gpt-oss:120b` and only use Graph of Thought on top of the provided code. The best policy (*003_dynamic_rrip_with_pc_based_bypass__v0.cc*) was generated using this at 0.408 IPC at a temperature of 1.

1) Success–Failure Distribution Across Temperatures: Across all workloads, each policy attempt was classified as either Success (valid policy produced and simulated) or Failure (compilation/parsing error).

Temperature	Failure	Success	Ratio
0.0	19	14	0.424242
0.5	8	47	0.854545
1.0	64	24	0.272727

TABLE II
SUCCESS-FAILURE ACROSS VARIOUS TEMPERATURES

At lower temperatures, models produced fewer failures and exhibited more stable behavior. At higher temperatures, the number of failures increased consistently, indicating greater variance and a tendency to produce syntactically invalid or logically inconsistent policies. The success ratio decreased monotonically with temperature, suggesting that more randomness negatively impacts policy correctness for this task. Overall, the error-progression plots show that most failures accumulate early in the run, stabilizing in later iterations as the system moves into exploitation and reuses previously successful patterns.

2) Error Progression Over Time: Figures 2, 3 and 4 shows error progression across each workload for all the temperatures. The plots show parsing errors, compilation errors, and cumulative errors across each attempt in policy generation. The error trend shows that:

1) Early iterations produce most errors, especially while generating sub-policies.
2) Once a general policy emerges, exploitation mode significantly reduces new errors.
3) This supports the design goal: the system becomes more stable as policy knowledge accumulates.

3) IPC Behavior Across Iterations: IPC trends were plotted separately for each temperature to isolate behavioral differences. The generated IPC was compared against the baseline IPC policy. Due to long queue times and GOT taking extra time due to generating a larger number of policies compared to a normal iteration, the number of iterations here is significantly less.

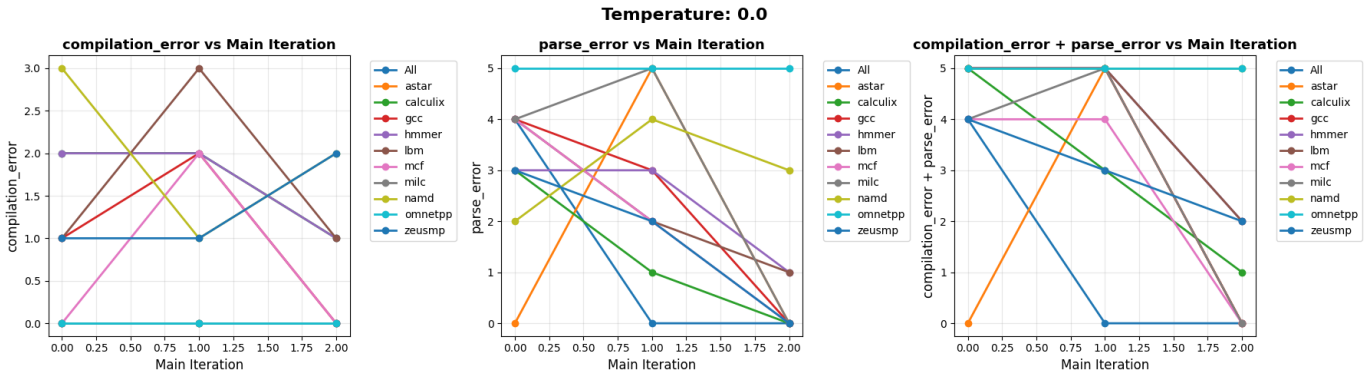

Fig. 2. Errors in policy generation for each workload across iterations at Temperature 0.0

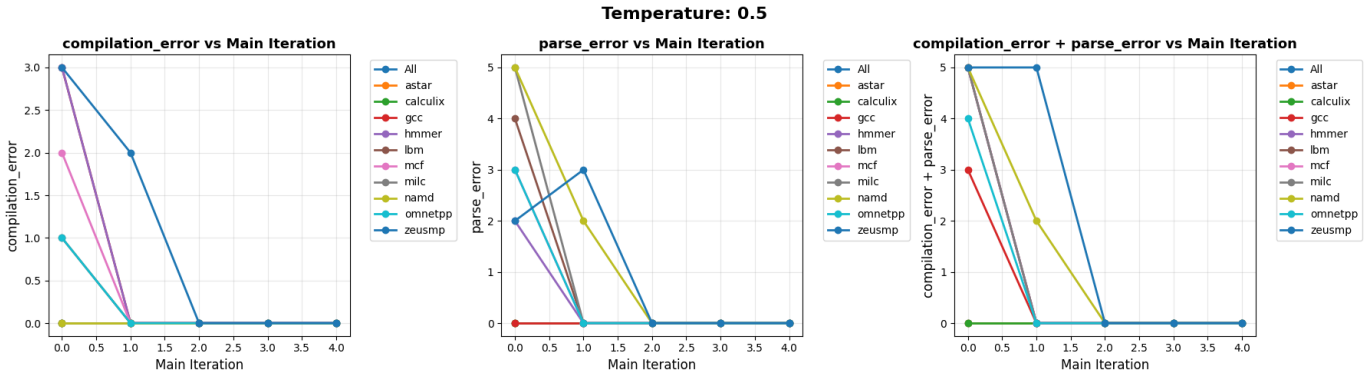

Fig. 3. Errors in policy generation for each workload across iterations at Temperature 0.5

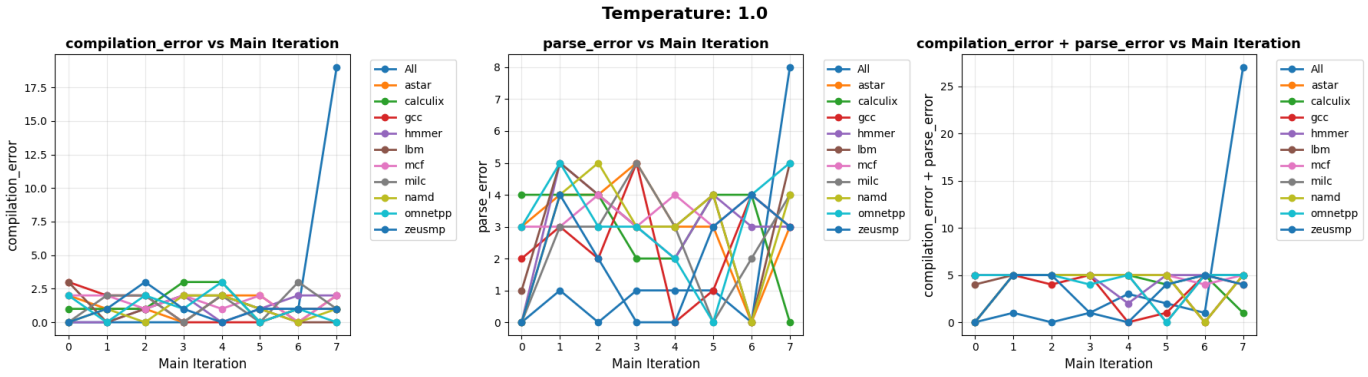

Fig. 4. Errors in policy generation for each workload across iterations at Temperature 1.0

Figures 5, 6 and 7 shows average IPC of combined policy across the iterations. While Figures 8, 9 and 10 give a much more per-workload analysis of the IPC across iterations. Based on the plots, following trends were observed.

1) **General trend:** For nearly all workloads, IPC improves over successive iterations. This indicates that iterative prompting and refinement help steer the model toward more efficient policies. The slope of improvement is steepest in the first few iterations, flattening afterward—consistent with diminishing returns as the policy space converges.
2) **Per-temperature differences:** IPC curves at temperature = low show smooth, monotonic increases, reflecting stable incremental refinement. Mid-temperature runs show more oscillation, but still trend upward. High-temperature runs produce irregular and noisy IPC trajectories, confirming that exploration overwhelms exploitation and consistency.
3) **Workload-level observations:** No strong correlation emerged between workload type and IPC, confirming that the framework performs uniformly across diverse workloads. Workloads requiring longer execution paths generally exhibit lower starting IPC but improve similarly across iterations.

4) Relationship Between IPC and Temperatures: The table III provides metrics on IPC stats per workload. While the table

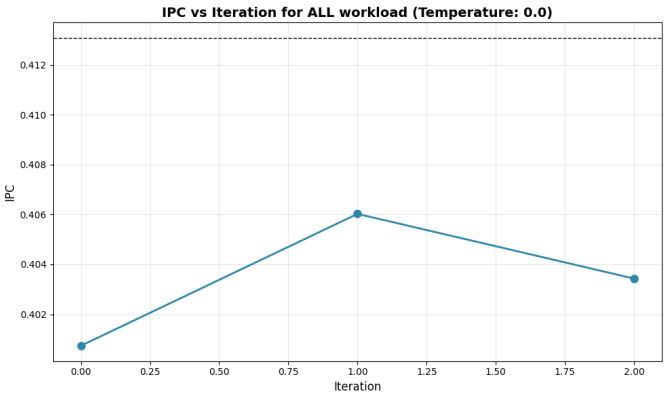

Fig. 5. Average IPC across iterations at Temperature 0.0

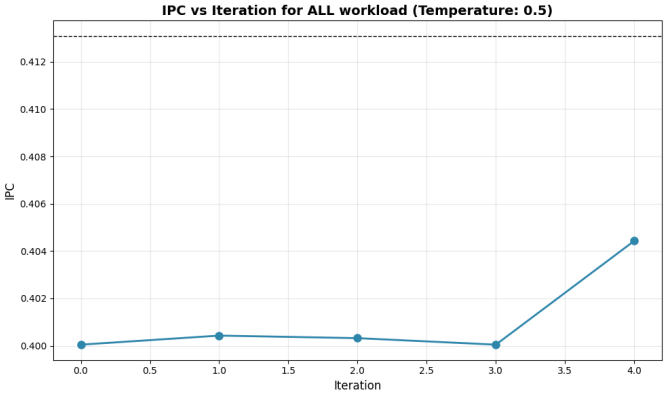

Fig. 6. Average IPC across iterations at Temperature 0.5

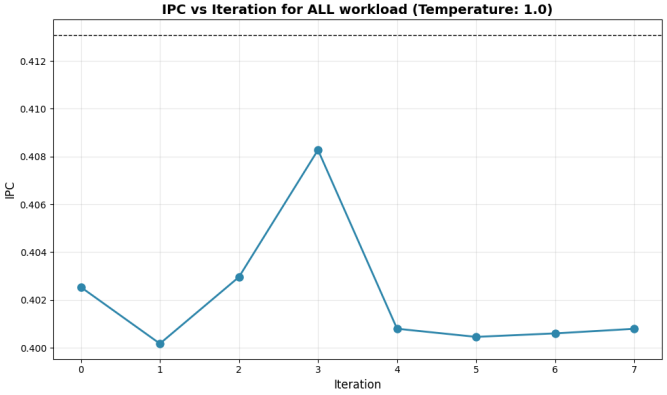

Fig. 7. Average IPC across iterations at Temperature 1.0

IV shows a correlation between IPC and the temperature of the model. We can see that there is a slight negative correlation between IPC and the temperature of the model if compared overall workloads, but some workloads have an advantage with higher temperatures of the model producing policies with greater IPC.

5) Exploration vs Exploitation: The figures 11, 12 and 13 show the behaviour of the model and it's switching between exploration and exploitation mode. A lot more iterations are needed to get a proper correlation between them.

In summary, low to mid temperature values offer the best

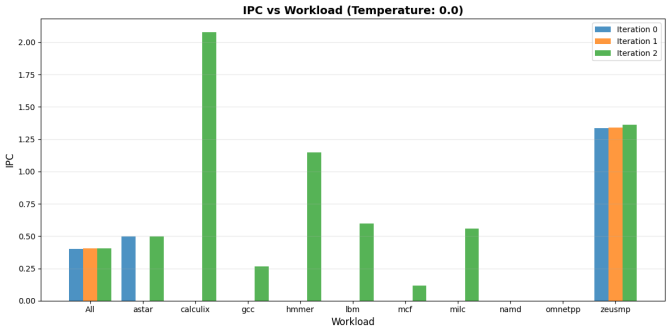

Fig. 8. Average IPC across iterations at Temperature 0.0 per workload

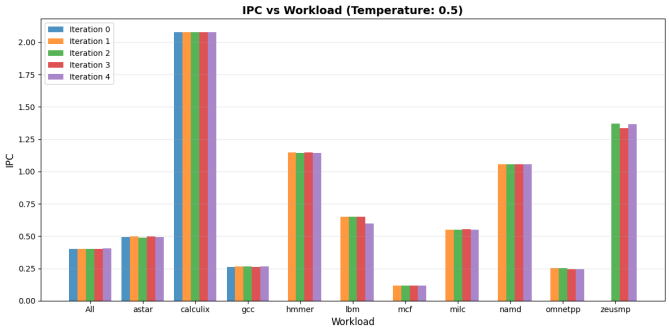

Fig. 9. Average IPC across iterations at Temperature 0.5 per workload

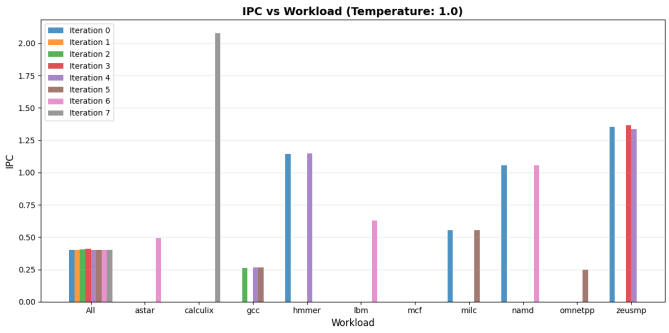

Fig. 10. Average IPC across iterations at Temperature 1.0 per workload

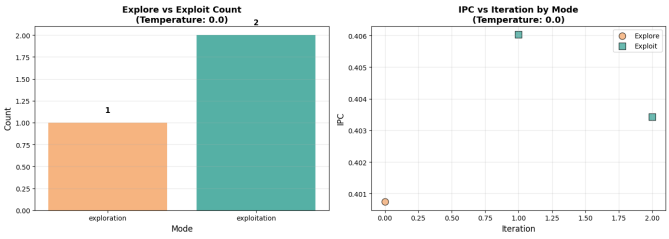

Fig. 11. Exploration and Exploitation at Temperature 0.0

trade-off between exploration and policy reliability. Although having higher temperatures results in generating unstable policies, they have the potential to generate more out-of-the-box policies, which can boost IPC significantly. Hence, implementing a system that changes the temperature based on mode (exploration/exploitation) will result in obtaining a policy that performs well as well as is stable.

Temperature	Workload	count	mean	min	max
0.0	All	3	0.403397	0.400736	0.406026
	astar	2	0.495420	0.495145	0.495694
	calculix	1	2.078060	2.078060	2.078060
	gcc	1	0.264888	0.264888	0.264888
	hmmer	1	1.146020	1.146020	1.146020
	lbm	1	0.595248	0.595248	0.595248
	mcf	1	0.116336	0.116336	0.116336
	milc	1	0.559163	0.559163	0.559163
	zeusmp	3	1.346493	1.337020	1.363600
0.5	All	5	0.401055	0.400047	0.404429
	astar	5	0.493076	0.488349	0.495686
	calculix	5	2.078060	2.078060	2.078060
	gcc	5	0.264464	0.262666	0.266769
	hmmer	4	1.145650	1.144260	1.146410
	lbm	4	0.635752	0.598887	0.648485
	mcf	4	0.117010	0.115933	0.117483
	milc	4	0.549401	0.547319	0.554013
	namd	4	1.056140	1.056140	1.056140
	omnetpp	4	0.249003	0.244969	0.252762
	zeusmp	3	1.356880	1.334540	1.369140
1.0	All	8	0.402075	0.400174	0.408276
	astar	1	0.491544	0.491544	0.491544
	calculix	1	2.078060	2.078060	2.078060
	gcc	3	0.265420	0.262668	0.266848
	hmmer	2	1.145620	1.145160	1.146080
	lbm	1	0.629819	0.629819	0.629819
	milc	2	0.552868	0.551571	0.554166
	namd	2	1.056140	1.056140	1.056140
	omnetpp	1	0.246581	0.246581	0.246581
	zeusmp	3	1.352600	1.337940	1.367520

TABLE III
IPC STATS ACROSS WORKLOADS AND TEMPERATURE

Workload	Correlation
All	-0.5628336282640172
astar	-0.9927575071503432
calculix	0.0
gcc	0.5551941321545721
hmmer	-0.8977249059234431
lbm	0.7902234116110116
mcf	1.0000000000000102
milc	-0.635982623350869
namd	0.0
omnetpp	-1.0
zeusmp	0.5849257820928476

TABLE IV
IPC TEMPERATURE CORRELATION ACROSS WORKLOADS

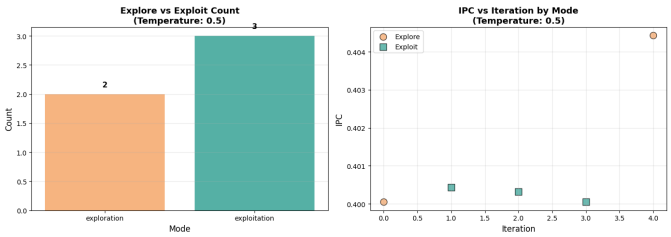

Fig. 12. Exploration and Exploitation at Temperature 0.5

C. Surrogate Model Integration Results

To evaluate the impact of the surrogate model for pre-screening policy candidates, we conducted experimental runs on both Ollama (open-source) and OpenAI (GPT-5) models integrated with Graph of Thought prompting and the surrogate reward predictor trained on over 2,000+ previous policies.

1) Ollama Performance with Surrogate Model: The Ollama model was run for 35 iterations with the surrogate model enabled, allowing the system to pre-score generated policies before expensive ChampSim simulation.

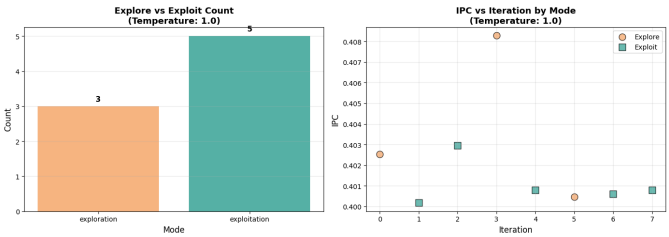

Fig. 13. Exploration and Exploitation at Temperature 1.0

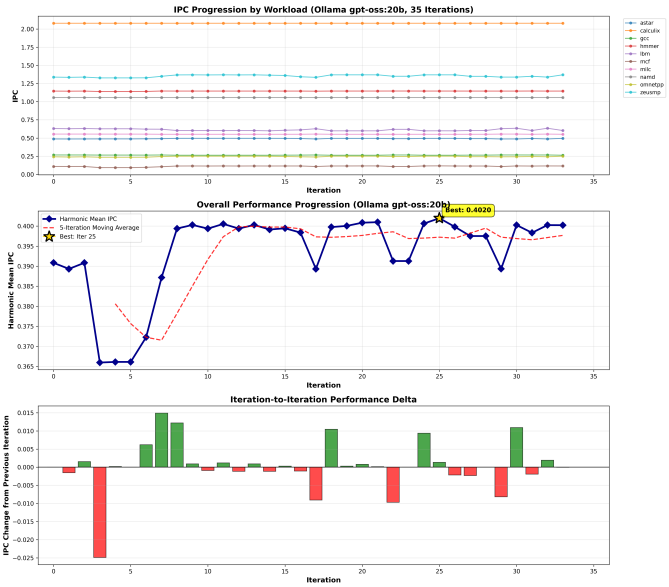

Fig. 14. Comprehensive IPC analysis for Ollama across 35 iterations with surrogate model

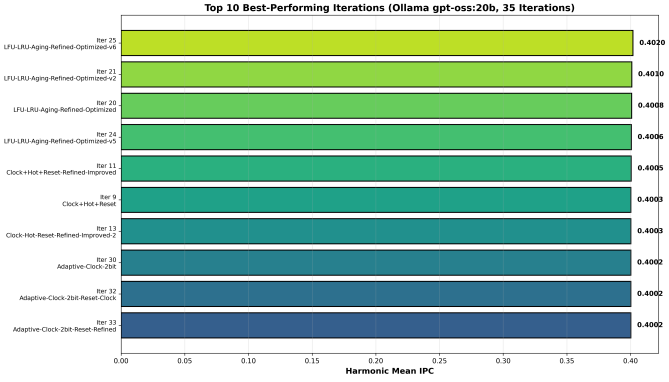

Fig. 15. Top-performing policies generated by Ollama with surrogate model

Figures 14, 15, 16, and 17 demonstrate the Ollama model's performance with surrogate integration. Key observations include:

1) The surrogate model enabled faster convergence by filtering low-quality candidates early in the pipeline.

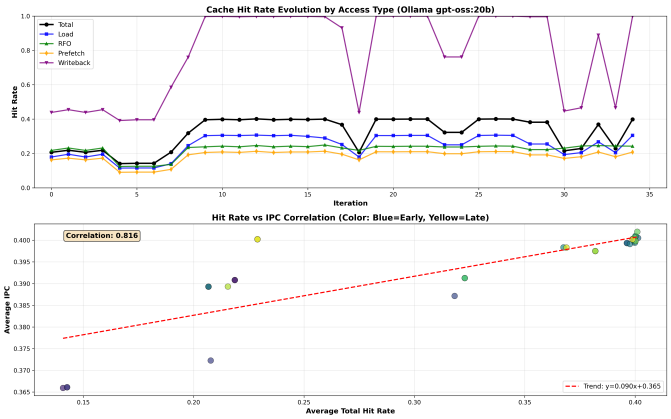

Fig. 16. Cache hit rate analysis across Ollama iterations

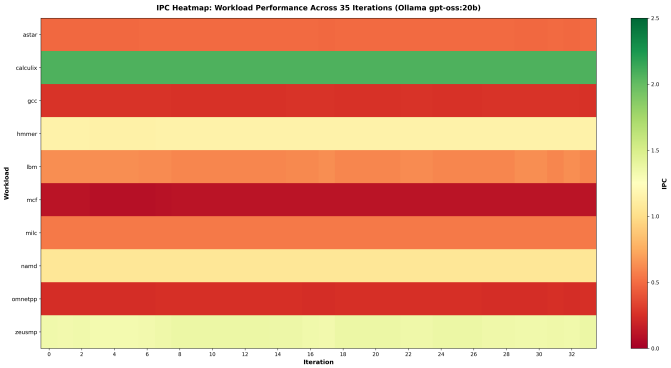

Fig. 17. Workload-specific performance heatmap for Ollama policies

2) IPC improvements showed steady progression across iterations, with the top policies achieving competitive performance.
3) Workload-specific analysis revealed varying effectiveness across different SPEC 2006 benchmarks.

2) OpenAI Performance with Surrogate Model: The OpenAI GPT-5 model was evaluated for 8 iterations with the same surrogate model configuration.

Figures 18, 19, 20, and 21 show the OpenAI model's performance. Despite fewer iterations due to API cost constraints, the results indicate:

1) OpenAI generated higher-quality initial policies, requiring fewer iterations to reach competitive IPC.
2) The surrogate model effectively prioritized promising candidates, reducing simulation overhead.
3) Hit rate patterns varied significantly across workloads, highlighting the importance of workload-aware policy design.

3) Comparative Analysis: Ollama vs. OpenAI: Comparing the two models with surrogate integration:

- **Sample Efficiency**: OpenAI achieved comparable performance in 8 iterations versus Ollama's 35, suggesting better initial policy quality.
- **Consistency**: Ollama showed more variability across iterations, while OpenAI maintained more stable IPC improvements.

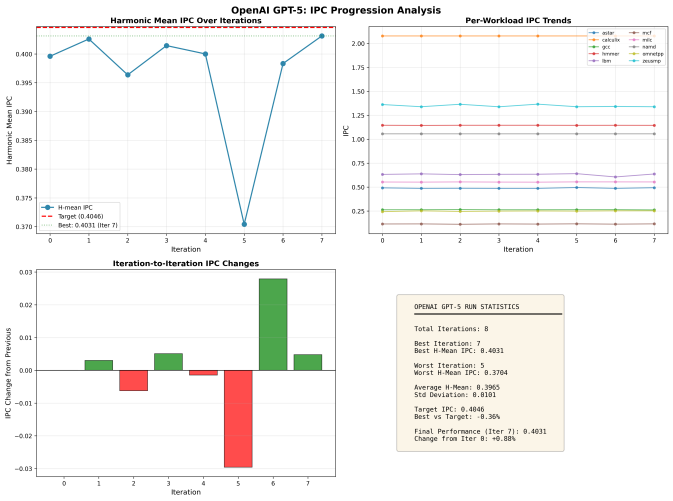

Fig. 18. Comprehensive IPC analysis for OpenAI across 8 iterations with surrogate model

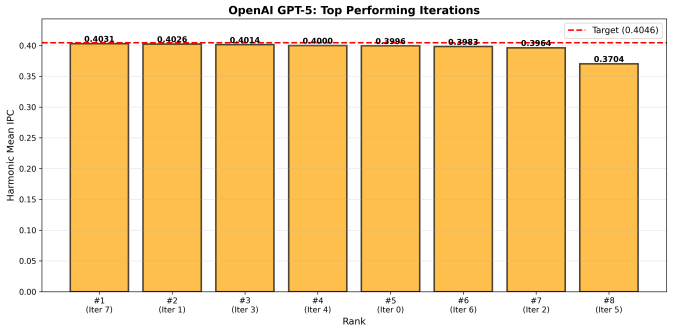

Fig. 19. Top-performing policies generated by OpenAI with surrogate model

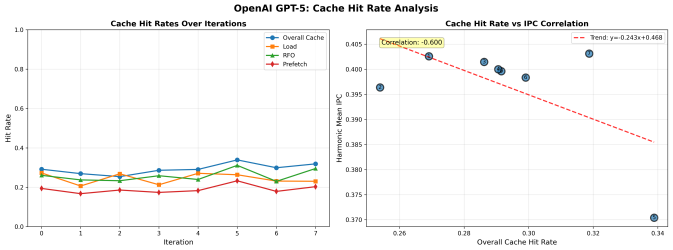

Fig. 20. Cache hit rate analysis across OpenAI iterations

- **Cost-Performance Trade-off**: While OpenAI required fewer iterations, the per-iteration API cost was significantly higher. Ollama provided a more cost-effective solution for extensive exploration.
- **Surrogate Model Impact**: Both models benefited from surrogate pre-screening, with reduced failed simulations and faster convergence compared to baseline runs without the surrogate.

D. Final Policy Performance Evaluation

We compared our best LLM-generated cache replacement policy with the baseline Hawkeye policy against all workloads through a standard bar plot in Figure 22 as well as the geometric mean speedup in Figure V below. From our observations,

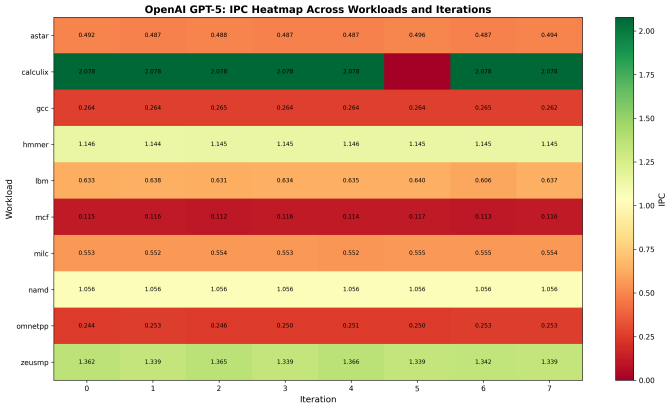

Fig. 21. Workload-specific performance heatmap for OpenAI policies

we can see some mild speedup (e.g. `gcc` workload and `milc` workload), similar performance (e.g. `calculix` workload and `namd` workload), and mild deprovement (e.g. `lbm` and `mcf` workload). These suggest that the policy showcasings strong handling of complex control flows and persistent cache lines, but suffers from mitigating cache pollution in streaming datasets from the baseline Hawkeye policy. These suggest that our policy maintained a lightweight local optimum, achieving near-parity (within 1%) of the baseline model with reduced hardware complexity, showcasing the efficacy of the LLM-in-the-loop framework that we have developed.

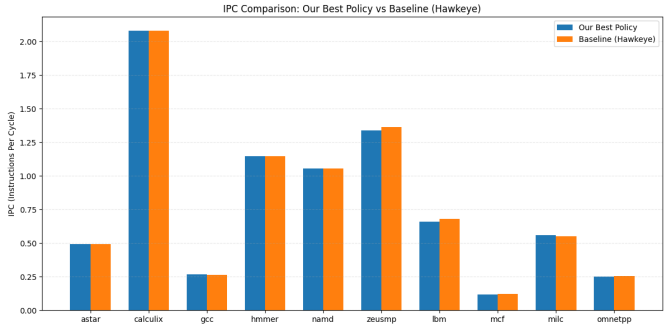

Fig. 22. IPC Comparison Bar Plot Between Hawkeye Baseline and Our Best Policy

TABLE V
IPC AND SPEEDUP COMPARISON: OUR BEST POLICY VS. BASELINE (HAWKEYE)

Workload	Our IPC	Baseline IPC	Speedup
astar	0.4909	0.4936	0.9947
calculix	2.0780	2.0780	1.0000
gcc	0.2685	0.2654	1.0119
hmmer	1.1464	1.1458	1.0005
namd	1.0561	1.0561	1.0000
zeusmp	1.3374	1.3617	0.9821
lbm	0.6595	0.6810	0.9685
mcf	0.1191	0.1225	0.9720
milc	0.5592	0.5522	1.0127
omnetpp	0.2512	0.2541	0.9886
Geometric Mean	**0.5874**	**0.5916**	**0.9930**

E. Area Table

Table VI shows the area calculation of the best policy generated: *003_dynamic_rrip_with_pc_based_bypass__v0.cc*

TABLE VI
AREA TABLE FOR DYNAMIC RRIP WITH PC-BASED BYPASS

Structure	Entries	Bits/entry	Bytes
RRPV (3b/line)	32,768	3	12,288
PC predictor table	1,024	5	640
Total		**103,424 b**	**≈ 12.63 KiB**

VII. CONCLUSION

ACKNOWLEDGMENTS

We thank our professor Dr. Samira Ajorpaz for providing us the opportunity to learn about generative AI designs, techniques, and methodologies and apply them towards solving scientific problems like cache replacement policy. We also thank the teaching assistants (Azam Ghanbari, Kaushal Mhapheskar, Tyler Williams, and Brijesh Bhanyana) of the course for their relentless efforts and help towards solving our group's (and others') questions when necessary.

CONTRIBUTIONS

Below, we have briefly described the individual contributions by each member of the project.

- Varad Patwardhan: I implemented the graph of thought system by splitting the workloads into sub-iterations and then generating and evaluating policies for each of them to combine and generate a final all-workload policy. I conducted a temperature sensitivity analysis using GPT-OSS:120B model on various temperatures and compared various statistics between them. I also contributed to various sections of the report. I contributed to the GOT section in the related work and methodology section, explaining the GOT paper and our implementation regarding that. I also worked on a report on the sensitivity analysis of the models and the area table calculation. Finally, I generated the reproduce.sh script for testing our script.
- David Mond: I implemented the surrogate model system for pre-scoring LLM-generated cache policy candidates, including the Ridge regression predictor, database integration, and package structure. I ran experimental iterations on both Ollama and OpenAI models using the surrogate model with Graph of Thought prompting, generating our results. I created comprehensive data visualization scripts and plots for analyzing performance across iterations, workloads, and policy comparisons. I helped implement our DAG lineage graph, and created our 6 checkpoints along the Evolutionary Path Analysis for Graph of Thought Cache Policy Discovery. I contributed to multiple branches on our repository including dmmond, got-implementation, and surrogate-model. Additionally, I contributed extensively to multiple sections of the report, including Design + Intelligent Search Strategy, Related Works, Automated Prompt Engineering (APE),

the Surrogate Model methodology, results for Surrogate + GoT, Evolutionary Path Analysis, and other key technical sections.

- Obblivignes KV: I contributed heavily towards all sections of the report, documenting relevant literature (including workloads and policy descriptions, among others), formatting and describing relevant results, and adding citations and references whenever required. I worked with Jaxon towards designing Automated Prompt Engineering and considered integrating it into our project through the `APE` branch on the GitHub repository. Finally, I helped lead other team members with setting up our repository and environments on the HPC cluster as well as assisting all other team members whenever needed in the Discord chat, which we used as our primary communication method.
- Jaxon Godbey: I contributed to designing the overall workflow for our system and proposed the idea of integrating Automated Prompt Engineering (APE) into our pipeline. I also helped figure out how APE could be implemented within our existing model to make policy generation more structured and reliable. In addition to the technical work, I helped write and refine several parts of the report and shaped how we described the system's process and methodology. Vignes and I also did the APE branch.
- Inyene Etuk: I designed and implemented the policy explainer pipeline that automatically translates raw ChampSim C++ replacement policies into structured, human-readable JSON summaries. This included building a robust OpenAI/Ollama integration layer, adding brace-based JSON extraction and error handling, and running the explainer across all baseline and generated policies for our experiments. I also contributed to the Related Works section by connecting our policy-interpretation approach to prior work on explainable systems and LLM-based code understanding, helping to frame the policy explainer as a key interpretability and debugging component within our overall workflow.
- Ravi Pavuluri: I worked with David to design the surrogate model system for pre-scoring LLM-generated cache policy candidates, assisted with writing one of the checkpoints for the evolutionary path analysis, and worked on the Related Works Section of the report.

REFERENCES

[1] A. Jaleel, K. B. Theobald, S. C. Steely, and J. Emer, "High performance cache replacement using re-reference interval prediction (RRIP)," *ACM SIGARCH Computer Architecture News*, vol. 38, no. 3, pp. 60–71, Jun. 2010. [Online]. Available: https://dl.acm.org/doi/10.1145/1816038.1815971

[2] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely, and J. Emer, "SHiP: signature-based hit predictor for high performance caching," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*. Porto Alegre Brazil: ACM, Dec. 2011, pp. 430–441. [Online]. Available: https://dl.acm.org/doi/10.1145/2155620.2155671

[3] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, no. 7995, pp. 468–475, Jan. 2024. [Online]. Available: https://www.nature.com/articles/s41586-023-06924-6

[4] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, "Graph of Thoughts: Solving Elaborate Problems with Large Language Models," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, pp. 17 682–17 690, Mar. 2024, arXiv:2308.09687 [cs]. [Online]. Available: http://arxiv.org/abs/2308.09687

[5] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large Language Models Are Human-Level Prompt Engineers," Mar. 2023, arXiv:2211.01910 [cs]. [Online]. Available: http://arxiv.org/abs/2211.01910

[6] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, "Adaptive insertion policies for high performance caching," *SIGARCH Comput. Archit. News*, vol. 35, no. 2, pp. 381–391, Jun. 2007. [Online]. Available: https://dl.acm.org/doi/10.1145/1273440.1250709

[7] A. Jain and C. Lin, "Back to the future: leveraging Belady's algorithm for improved cache replacement," *SIGARCH Comput. Archit. News*, vol. 44, no. 3, pp. 78–89, Jun. 2016. [Online]. Available: https://dl.acm.org/doi/10.1145/3007787.3001146

[8] Z. Shi, X. Huang, A. Jain, and C. Lin, "Applying Deep Learning to the Cache Replacement Problem," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. Columbus OH USA: ACM, Oct. 2019, pp. 413–425. [Online]. Available: https://dl.acm.org/doi/10.1145/3352460.3358319

[9] W. K. Chai, D. He, I. Psaras, and G. Pavlou, "Cache "Less for More" in Information-Centric Networks," in *NETWORKING 2012*, R. Bestak, L. Kencl, L. E. Li, J. Widmer, and H. Yin, Eds. Berlin, Heidelberg: Springer, 2012, pp. 27–40.

[10] D. Jimenez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proceedings HPCA Seventh International Symposium on High-Performance Computer Architecture*, Jan. 2001, pp. 197–206. [Online]. Available: https://ieeexplore.ieee.org/document/903263/

[11] D. A. Jiménez and E. Teran, "Multiperspective reuse prediction," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. Cambridge Massachusetts: ACM, Oct. 2017, pp. 436–448. [Online]. Available: https://dl.acm.org/doi/10.1145/3123939.3123942

[12] S. Sethumurugan, J. Yin, and J. Sartori, "Designing a Cost-Effective Cache Replacement Policy using Machine Learning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, Feb. 2021, pp. 291–303. [Online]. Available: https://ieeexplore.ieee.org/document/9407137/

[13] H. J. Yoo, J. H. Kim, and T. H. Han, "RL-Based Cache Replacement: A Modern Interpretation of Belady's Algorithm With Bypass Mechanism and Access Type Analysis," *IEEE Access*, vol. 11, pp. 145 238–145 253, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10373004/

[14] "403.gcc: SPEC CPU2006 Benchmark Description." [Online]. Available: https://www.spec.org/cpu2006/Docs/403.gcc.html

[15] "429.mcf: SPEC CPU2006 Benchmark Description." [Online]. Available: https://www.spec.org/cpu2006/Docs/429.mcf.html

[16] "433.milc: SPEC CPU2006 Benchmark Description." [Online]. Available: https://www.spec.org/cpu2006/Docs/433.milc.html

[17] "434.zeusmp: SPEC CPU2006 Benchmark Description." [Online]. Available: https://www.spec.org/cpu2006/Docs/434.zeusmp.html

[18] "444.namd: SPEC CPU2006 Benchmark Description." [Online]. Available: https://www.spec.org/cpu2006/Docs/444.namd.html

[19] "454.calculix: SPEC CPU2006 Benchmark Description." [Online]. Available: https://www.spec.org/cpu2006/Docs/454.calculix.html

[20] "456.hmmer: SPEC CPU2006 Benchmark Description." [Online]. Available: https://www.spec.org/cpu2006/Docs/456.hmmer.html

[21] "470.lbm: CPU2006 Benchmark Description." [Online]. Available: https://www.spec.org/cpu2006/Docs/470.lbm.html

[22] "471.omnetpp: SPEC CPU2006 Benchmark Description." [Online]. Available: https://www.spec.org/cpu2006/Docs/471.omnetpp.html

[23] "473.astar: SPEC CPU2006 Benchmark Description." [Online]. Available: https://www.spec.org/cpu2006/Docs/473.astar.html

[24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," Aug. 2023, arXiv:1706.03762 [cs]. [Online]. Available: http://arxiv.org/abs/1706.03762

[25] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models From Natural

Language Supervision," Feb. 2021, arXiv:2103.00020 [cs]. [Online]. Available: http://arxiv.org/abs/2103.00020
[26] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," Jan. 2015, arXiv:1409.0575 [cs]. [Online]. Available: http://arxiv.org/abs/1409.0575
[27] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," Jan. 2023, arXiv:2201.11903 [cs]. [Online]. Available: http://arxiv.org/abs/2201.11903
[28] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus, "Emergent Abilities of Large Language Models," Oct. 2022, arXiv:2206.07682 [cs]. [Online]. Available: http://arxiv.org/abs/2206.07682
[29] G. Chen, S. Dong, Y. Shu, G. Zhang, J. Sesay, B. F. Karlsson, J. Fu, and Y. Shi, "AutoAgents: A Framework for Automatic Agent Generation," Apr. 2024, arXiv:2309.17288 [cs]. [Online]. Available: http://arxiv.org/abs/2309.17288
[30] A. T. Kalai, O. Nachum, S. S. Vempala, and E. Zhang, "Why Language Models Hallucinate," Sep. 2025, arXiv:2509.04664 [cs]. [Online]. Available: http://arxiv.org/abs/2509.04664
[31] C. Mendis, A. Renda, S. Amarasinghe, and M. Carbin, "Ithemal: Accurate, Portable and Fast Basic Block Throughput Estimation using Deep Neural Networks," May 2019, arXiv:1808.07412 [cs]. [Online]. Available: http://arxiv.org/abs/1808.07412
[32] P. Shojaee, I. Mirzadeh, K. Alizadeh, M. Horton, S. Bengio, and M. Farajtabar, "The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity," Nov. 2025, arXiv:2506.06941 [cs]. [Online]. Available: http://arxiv.org/abs/2506.06941
[33] Y. Wu, T. Cooijmans, K. Kastner, A. Roberts, I. Simon, A. Scarlatos, C. Donahue, C. Tarakajian, S. Omidshafiei, A. Courville, P. S. Castro, N. Jaques, and C.-Z. A. Huang, "Adaptive Accompaniment with ReaLchords," Jun. 2025, arXiv:2506.14723 [cs]. [Online]. Available: http://arxiv.org/abs/2506.14723
[34] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," Mar. 2023, arXiv:2210.03629 [cs]. [Online]. Available: http://arxiv.org/abs/2210.03629
[35] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," Jun. 2025, arXiv:2506.13131 [cs]. [Online]. Available: http://arxiv.org/abs/2506.13131

APPENDIX

DELIVERABLES

1. Best performing Replacement Policy code.

Listing 1. Best Policy C++ Code

```cpp
#include <cstdint>
#include "../inc/champsim_crc2.h"

#define NUM_CORE 1
#define LLC_SETS (NUM_CORE * 2048)
#define LLC_WAYS 16

static const uint32_t MAX_RRPV = 7;           // 3 bit RRIP
static uint8_t rrpv[LLC_SETS][LLC_WAYS];      // per line RRIP counters

// PC predictor: 1024 entries, 5 bit saturating counters
static const uint32_t PC_TABLE_SIZE = 1024;
static uint8_t pc_counter[PC_TABLE_SIZE];   // only low 5 bits used
static const uint8_t PC_MAX = 31;
static const uint8_t PC_MIN = 0;
static const uint8_t PC_THRESHOLD = 8;       // below = cold, bypass

inline uint32_t pc_index(uint64_t pc) {
    // simple xor hash, power of two size
    return (uint32_t)((pc ^ (pc >> 12)) & (PC_TABLE_SIZE - 1));
}

void InitReplacementState() {
    for (uint32_t s = 0; s < LLC_SETS; ++s) {
        for (uint32_t w = 0; w < LLC_WAYS; ++w)
            rrpv[s][w] = MAX_RRPV;   // start as most distant
    }
    for (uint32_t i = 0; i < PC_TABLE_SIZE; ++i)
        pc_counter[i] = PC_THRESHOLD; // neutral start
}

uint32_t GetVictimInSet(uint32_t cpu, uint32_t set, const BLOCK *current_set,
                        uint64_t PC, uint64_t paddr, uint32_t type) {
    // 1) Return any invalid way immediately
    for (uint32_t w = 0; w < LLC_WAYS; ++w) {
        if (!current_set[w].valid) return w;
    }

    // 2) Bypass decision (allowed for LOAD/RFO/PREFETCH only)
    // In ChampSim, type values: 0=LOAD, 1=RFO, 2=PREFETCH, 3=WRITEBACK
    if (type != 3) { // not a WRITEBACK
        uint32_t idx = pc_index(PC);
        if (pc_counter[idx] < PC_THRESHOLD) {
            return LLC_WAYS; // indicate bypass
        }
    }

    // 3) RRIP victim search
    while (true) {
        for (uint32_t w = 0; w < LLC_WAYS; ++w) {
            if (rrpv[set][w] == MAX_RRPV) return w;
        }
        // Increment all counters that are not already max
        for (uint32_t w = 0; w < LLC_WAYS; ++w) {
            if (rrpv[set][w] < MAX_RRPV) rrpv[set][w]++;
        }
    }
    // Unreachable
    return 0;
}

void UpdateReplacementState(uint32_t cpu, uint32_t set, uint32_t way,
                            uint64_t paddr, uint64_t PC, uint64_t victim_addr,
                            uint32_t type, uint8_t hit) {
    // Ignore writebacks updates
    if (type == 3) return;

    uint32_t idx = pc_index(PC);

    if (hit) {
        // Hit: promote to MRU
        rrpv[set][way] = 0;
        // Increment PC counter (saturating)
```

```
        if (pc_counter[idx] < PC_MAX) pc_counter
            [idx]++;
    } else {
        // Miss (fill): insert with RRPV =
            MAX_RRPV-1
        rrpv[set][way] = MAX_RRPV - 1;
        // Decrement PC counter (saturating)
        if (pc_counter[idx] > PC_MIN) pc_counter
            [idx]--;
    }
}

// Keep stats functions empty as required
void PrintStats() {}
void PrintStats_Heartbeat() {}
```

2. **GitHub Repository.**
   - The link to our public GitHub repository with a detailed README.md can be found here: https://github.com/dmmond_ncstate/CacheForgeProject

3. **OpenAI API Expense Logs**

TABLE VII
OPENAI API USAGE DETAILS

Unity ID	Date	Cost (USD)
dmmond	Dec 7, 2025	1.972
dmmond	Dec 8, 2025	10.232
	Total:	**12.204**

4. **OS, CPU model, RAM;** compiler version (gcc or Clang). For all of our experiments and simulations, we utilized the HPC cluster at NC State, which provided us with the following configurations.
   - Operating System: Linux
   - CPU Model: Intel(R) Xeon(R) Gold 6226R CPU @ 2.90GHz
   - LLM: Ollama's gpt-oss:120b & OpenAI's gpt-5.1
   - RAM: 187 GB
   - gcc: 11.5

5. **ChampSim commit/branch;** CacheForge commit; any submodules. We used the default ChampSim and CacheForge configurations/commit/branch as provided by the course instructor.

6. **Random seeds and number of trials per point.** Used random seeds between 1 to 1000000 and tested all iterations as described in report due to resource and time constraints.

7. **Exact commands used to produce each figure/table.**
   - Table I: Derived values from running `python DB_connection.py` provided in CacheForge and compiled manually.
   - Figure 1: Figure was generated using the appropriate script in the `analysis` folder of the `dmmond` branch.
   - Table II: Table was derived from `Plots/plot.ipynb` inside the `main` branch.
   - Figures 2, 3, 4: All figures were derived from `Plots/plot.ipynb` inside the `main` branch.
   - Figures 5, 6, 7: All figures were derived from `Plots/plot.ipynb` inside the `main` branch.
   - Figures 8, 9, 10: All figures were derived from `Plots/plot.ipynb` inside the `main` branch.
   - Figures 11, 12, 13: Figure was derived from `Plots/plot.ipynb` inside the `main` branch.
   - Table III: Table was derived from `Plots/plot.ipynb` inside the `main` branch.
   - Table IV: Table was derived from `Plots/plot.ipynb` inside the `main` branch.
   - Figures 14, 15, 17, 16: All Ollama figures were generated using the respective scripts in the `analysis` folder under the `dmmond` branch.
   - Figures 18, 19, 20, 21: All OpenAI figures were generated using the respective scripts in the `analysis` folder under the `dmmond` branch.
   - Figure 22: Figure was generated from the `Plots/ipc_comparison.py` file in the `main` branch.
   - Table V: Table was generated from the `Plots/ipc_comparison.py` file in the `main` branch.
   - Table VI: Area table was calculated from the design of the LLM-generated policy in Listing 1.
   - Table VII: Derived from OpenAI Platform generated spreadsheet.
   - Listing 1: Best C++ LLM-generated Cache Replacement policy
   - Listing 2: Derived from running `pip freeze` in the current environment.

8. **Python version;** exact `requirements.txt` lock (e.g., `pip freeze`). The exact `pip freeze` output used in our environment for all experiments, simulations, and measurements can be viewed in Listing 2 below. All lineplots and barplots used matplotlib and were run on a local environment, which used the following `pip freeze` output: `matplotlib==3.9.4`.

Listing 2. Pip freeze output

```
altgraph==0.17.4
annotated-types==0.7.0
anyio==4.12.0
argcomplete==1.12.0
Babel==2.9.1
bcc==0.32.0
certifi==2025.11.12
cffi==1.14.5
chardet==4.0.0
cockpit @ file:///builddir/build/BUILD/cockpit-334.1/tmp/wheel/cockpit-334.1-py3-none-any.whl
configshell-fb==1.1.28
contourpy==1.3.0
cryptography==36.0.1
cycler==0.12.1
dasbus==1.4
dbus-python==1.2.18
decorator==4.4.2
distro==1.9.0
dnspython==2.6.1
ethtool==0.15
exceptiongroup==1.3.1
fail2ban==1.0.2
file-magic==0.4.0
fonttools==4.60.1
gpg==1.15.1
gssapi==1.6.9
h11==0.16.0
httpcore==1.0.9
httpx==0.28.1
idna==2.10
importlib_metadata==8.7.0
importlib_resources==6.5.2
iniparse==0.4
iotop==0.6
ipaclient==4.12.2
ipalib==4.12.2
ipaplatform==4.12.2
ipapython==4.12.2
Jinja2==2.11.3
jiter==0.12.0
joblib==1.5.2
jsonpointer==2.0
jwcrypto==1.5.6
kiwisolver==1.4.7
kmod==0.1
ldap3==2.9.1
libcomps==0.1.18
libvirt-python==9.0.0
lxml==4.6.5
MarkupSafe==1.1.1
matplotlib==3.9.4
mysql-connector-python==9.4.0
netaddr==0.8.0
netifaces==0.10.6
nftables==0.1
numpy==2.0.2
ollama==0.6.1
openai==2.9.0
packaging==25.0
pcp==5.0
perf==0.1
pexpect==4.8.0
pillow==11.3.0
ply==3.11
psutil==5.8.0
psycopg2==2.8.6
ptyprocess==0.6.0
pyasn1==0.6.1
pyasn1-modules==0.2.8
pycparser==2.20
pydantic==2.12.5
pydantic_core==2.41.5
pyelftools==0.27
PyGObject==3.40.1
pyinotify==0.9.6
pyinstaller==6.14.2
pyinstaller-hooks-contrib==2025.8
pyodbc===4.0.0-unsupported
pyparsing==2.4.7
PySocks==1.7.1
python-augeas==0.5.0
python-dateutil==2.8.1
python-ldap==3.4.3
python-linux-procfs==0.7.0
python-yubico==1.3.3
pytz==2021.1
pyudev==0.22.0
pyusb==1.0.2
PyYAML==5.4.1
qrcode==6.1
requests==2.25.1
rpm==4.16.1.3
rtslib-fb==2.1.75
scikit-learn==1.6.1
scipy==1.6.2
selinux==3.5
sepolicy==3.5
setools==4.4.1
setroubleshoot==3.3.31
six==1.15.0
sniffio==1.3.1
sos==4.5.1
SSSDConfig==2.9.7
subscription-manager==1.29.45
systemd-python==234
targetcli-fb==2.1.53
threadpoolctl==3.6.0
tqdm==4.67.1
typing-inspection==0.4.2
typing_extensions==4.15.0
urllib3==1.26.5
urwid==2.1.2
zipp==3.23.0
```